

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Maik Poetzsch
	Sebastian Seipel	

2025-06-17

Inhaltsverzeichnis

Methodenbaustein Sensordatenanalyse	5
Voraussetzungen	6
Lernziele	7
1 Das Prinzip von Messungen	8
1.1 Messung	10
1.1.1 Direkte und indirekte Messung	10
1.1.2 Genauigkeit und Präzision	12
1.2 Messreihen	12
1.3 Varianz	12
1.4 Standardabweichung	13
1.4.1 Experiment Verteilungskenngrößen	14
1.5 Ergebnisse	15
1.6 grafische Darstellung	15
1.7 Code	16
1.7.1 Aufgabe Verteilungskenngrößen	18
1.7.2 Varianz und Standardabweichung mit NumPy und Pandas	20
2 Die Normalverteilung	22
2.1 ein Würfel	22
2.2 zwei Würfel	23
2.3 drei Würfel	25
2.4 Grafik	30
2.5 Code	30
2.6 absolute Häufigkeit	33
2.7 relative Häufigkeit	33
2.8 Häufigkeitsdichte	33
2.9 3 Klassen	33
2.10 5 Klassen	33
2.11 7 Klassen	33
2.12 Normalverteilung anpassen	34
2.13 Das Paket SciPy	36
2.14 Aufgaben Normalverteilung	38
2.15 Konfidenzintervalle	39

2.16	Standardnormalverteilung	40
2.17	Einzelner Messwert	41
2.18	Stichprobe $N = 12$	42
2.19	Code	42
2.20	Die t-Verteilung	44
2.21	Grafik	45
2.22	Code	46
2.23	Grafik	48
2.24	Code	48
2.25	Aufgabe Konfidenzintervalle	50
3	Lineare Parameterschätzung	52
3.1	Messreihe Hooke'sches Gesetz	52
3.1.1	Deskriptive Statistik	54
3.2	Federkonstante bestimmen	62
3.2.1	Lineare Ausgleichsrechnung	63
3.3	Grafik	64
3.4	Code	64
3.4.1	Messabweichung quantifizieren	70
3.4.2	Das Modul SciPy	73
3.4.3	Ergebnis Federkonstante	74
3.5	Resterampe	74
4	Fehlerrechnung	75
4.1	Fehlerarten und Umgang mit Messfehlern	75
4.2	Methoden zur Fehlerabschätzung	76
4.2.1	Fehlerabschätzung	77
5	Übung Hooke'sches Gesetz	79
5.1	Dateien einlesen	80
5.2	Aufgabe Dateien einlesen	81
5.3	Daten aufbereiten	89
5.3.1	Auf Ausreißer prüfen	89
5.4	Ausreißer bestimmen	89
5.5	Gemeinsame Ausgabe der Messreihen	90
5.6	Code	91
5.6.1	Umwandlung der Rechengrößen	93
5.6.2	Grafische Darstellung	95
5.7	Grafische Darstellung	97
5.8	Mittelwerte und Standardfehler berechnen	98
5.9	Code	98
5.10	Auswertung	99
5.11	Federkonstanten bestimmen	100

5.12	$\alpha = 0.10$	100
5.13	$\alpha = 0.05$	101
5.14	Code	101
5.15	Auswertung	102

Methodenbaustein Sensordatenanalyse



Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Maik Poetzsch und Sebastian Seipel. Methodenbaustein Sensordatenanalyse von Maik Poetzsch ist lizenziert unter [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Das Werk ist abrufbar auf [GitHub](https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse). Ausgenommen von der Lizenz sind alle Logos Dritter und anders gekennzeichneten Inhalte. 2025

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Maik Poetzsch, und Sebastian Seipel. 2025. „Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse. <https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse>.”

BibTeX-Vorlage

```
@misc{BCD-m-sensordatenanalyse-2025,  
  title={Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse},  
  author={Arnold, Lukas and Arnold, Simone and Bagemihl, Florian and Baitsch, Matthias and Fehr, Marc and Poetzsch, Maik and Seipel, Sebastian},  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse}}
```

Voraussetzungen

Die Bearbeitungszeit dieses Bausteins beträgt circa **Platzhalter**. Für die Bearbeitung dieses Bausteins werden folgende Bausteine vorausgesetzt und die genannten Bibliotheken verwendet:

- numpy
 - numpy.polynomial
- pandas
- matplotlib

Querverweis auf:

- w-python
- m-EsD

Im Baustein werden folgende Daten verwendet:

Lernziele

In diesen Baustein lernen Sie ...

- Statistische Grundbegriffe
- Werkzeuge zur Auswertung und grafischen Darstellung von Sensordaten kennen
- Methoden zur Verarbeitung von mehreren Dateien anzuwenden
- Fehlerrechnung und -analyse
- Sensorkennlinien
- Kennlinienfehler und deren Korrektur

1 Das Prinzip von Messungen

“In der Physik existiert nur das, was gemessen worden ist” (Merz 1968, 14).
Merz, Ludwig. 1968. “Grundkurs der Messtechnik. Teil I: Das Messen elektrischer Größen.” 2. Auflage. München; Wien. R. Oldenbourg Verlag.

to do: Größtfehler ergänzen

In diesem Baustein werden die folgenden Module verwendet:

```
import numpy as np
import numpy.polynomial.polynomial as poly
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import glob
```

Physikalische Größen werden mit der Hilfe von Messgeräten bestimmt. Diese ordnen der tatsächlichen Merkmalsausprägung eine numerische Entsprechung relativ zu einem Bezugssystem zu.

Ein Beispiel: “Johanna ist am Messbrett 173 Zentimeter groß.”

- Die tatsächliche Merkmalsausprägung ist Johannas Größe.
- Das Messgerät ist das Messbrett.
- Die numerische Entsprechung ist 173.
- Das Bezugssystem ist das metrische System.

Messwerte sind aus verschiedenen Gründen Annäherungen an den wahren Wert der zugrundeliegenden physikalischen Größe. Zum einen variiert die [Größe eines Menschen im Tagesverlauf](#). Zum anderen ist das Messergebnis auch ein Ergebnis der verwendeten Skala. Wäre die Messung im imperialen Messsystem erfolgt, wäre Johannas Größe mit 68 Zoll bestimmt worden, was 172,72 Zentimetern entspricht.

Das Messergebnis ist also keine exakte Entsprechung der tatsächlichen Merkmalsausprägung. Ein bekanntes Beispiel für die mit dem Messvorgang verbundene Unsicherheit ist das Küstenlinienparadox: Das Ergebnis der Vermessung unregelmäßiger Küstenlinien wird umso größer, je kleiner die Messabschnitte gewählt werden.



- (a) Gerade Messabschnitte von 200 km Länge, Gesamtlänge ungefähr 2350 km
 (b) Gerade Messabschnitte von 100 km Länge, Gesamtlänge ungefähr 2775 km
 (c) Gerade Messabschnitte von 50 km Länge, Gesamtlänge ungefähr 3425 km

Abbildung 1.1: Küstenlinienparadox

Britain-fractal-coastline-200km, Britain-fractal-coastline-100km und Britain-fractal-coastline-50km von Maksim stehen unter der Lizenz [CC BY-SA 3.0](#) und sind abrufbar auf Wikipedia ([200km](#), [100km](#), [50km](#)). 2006

1.1 Messung

! Wichtig 1: Messung

“Eine Messung ist der experimentelle Vorgang, durch den ein spezieller Wert einer physikalischen Größe als Vielfaches einer Einheit oder eines Bezugswertes ermittelt wird.

Die Messung ergibt zunächst einen Messwert. Dieser stimmt aber aufgrund störender Einflüsse mit dem wahren Wert der Messgröße praktisch nie überein, sondern weist eine gewisse Messabweichung auf. Zum *vollständigen Messergebnis* wird der Messwert, wenn er mit quantitativen Aussagen über die zu erwartende Größe der Messabweichung ergänzt wird. Dies wird in der Messtechnik als Teil der Messaufgabe und damit der Messung verstanden.”

Messung. von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia] (<https://de.wikipedia.org/wiki/Messung>). 2025

- Die ideale Messung ist eine direkte Messung oder der gesuchte Wert hängt linear (**direkt?!**) vom gemessenen Wert ab.
- Die ideale Messung ist *genau* und *präzise*.

1.1.1 Direkte und indirekte Messung

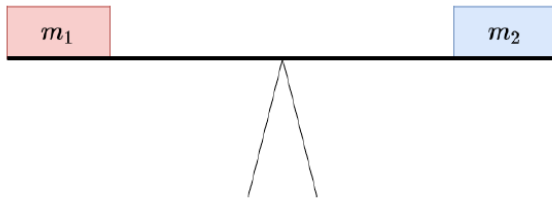
Bei einer direkten Messung wird die Messgröße durch den unmittelbaren Vergleich mit einem Normal oder einem genormten Bezugssystem gewonnen.

Gliedermaßstäbe von Fst76 ist lizenziert unter [CC-BY-SA 3.0](#) und ist abrufbar auf [Wikimedia](#). 2014

Bei einer indirekten Messung wird die Messgröße auf eine andere physikalische Größe zurückgeführt.

Spring scale von Amada44 steht unter der Lizenz [CC-BY-SA-3.0 unported](#) und ist abrufbar auf [Wikimedia](#). 2016

Observe the Moon wurde von der NASA veröffentlicht und ist abrufbar unter [nasa.gov](#). 2010



(a) Balkenwaage

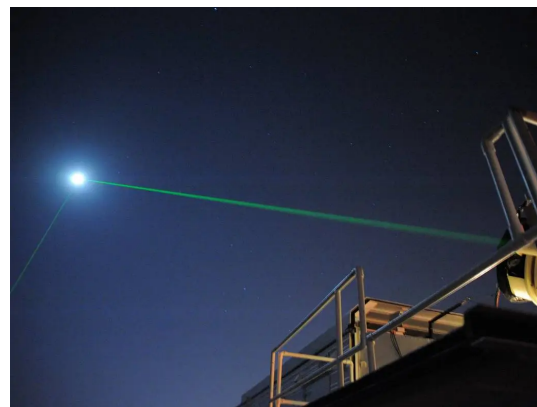


(b) Zollstock

Abbildung 1.2: Direkte Messung



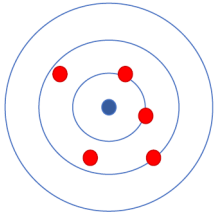
(a) Federwaage



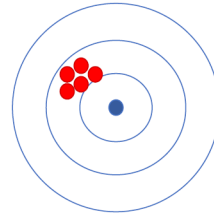
(b) Laserentfernungsmessung

Abbildung 1.3: Indirekte Messung

1.1.2 Genauigkeit und Präzision



(a) Genauigkeit



(b) Präzision

Die Genauigkeit einer Messung ist ein Maß für die Abweichung der Messwerte vom realen Wert. Die Genauigkeit ist nur bestimmbar, wenn anerkannte Referenzwerte vorhanden sind. Die Präzision einer Messung beschreibt, wie gut die einzelnen Messwerte miteinander übereinstimmen. Die Präzision einer Messung wird über die Standardabweichung der Stichprobe bestimmt.

Abbildung 1.4: Genauigkeit und Präzision

1.2 Messreihen

Um die Unsicherheit einer Messung zu verringern, kann man einen Messwert in Form einer Messreihe wiederholt aufnehmen. Die (erste) beste Schätzung der Messgröße bietet der arithmetische Mittelwert der Messreihe.

Der arithmetische Mittelwert einer Messreihe $\bar{x}$ ist die Summe aller Einzelmesswerte dividiert durch die Anzahl der Messwerte N .

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Mit Hilfe des arithmetischen Mittelwerts kann eine Aussage über die Streuung der Messwerte und die Präzision der Messung getroffen werden. Dazu werden die Varianz und die Standardabweichung der Messreihe berechnet.

1.3 Varianz

Die Varianz ist der Mittelwert der quadrierten Abweichungen vom Mittelwert.

$$\text{Var}(x_i) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

1.4 Standardabweichung

Die Quadratwurzel der Varianz wird als Standardabweichung bezeichnet. Diese hat den Vorteil, dass sie in der Einheit der Messwerte vorliegt und dadurch leichter zu interpretieren ist. Die Standardabweichung s wird so berechnet:

$$s_N = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Für Stichproben wird die Stichprobenvarianz verwendet. Für die Standardabweichung einer Stichprobe gilt:

$$s_{N-1} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Da die Varianz das Quadrat der Standardabweichung s ist, wird diese häufig mit s^2 gekennzeichnet.

Warning 1: Standardabweichung und Varianz in der Grundgesamtheit

In der Stochastik werden Formeln häufig auch mit griechischen Buchstaben geschrieben, wenn Sie sich statt auf eine Stichprobe auf die Grundgesamtheit beziehen.

Der Mittelwert in der Grundgesamtheit wird auch Erwartungswert genannt und mit dem griechischen Buchstaben μ (My) dargestellt. Die Standardabweichung des Erwartungswerts wird mit σ (Sigma) gekennzeichnet.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

Mit Hilfe der Standardabweichung kann der *Standardfehler* der Messung bestimmt werden. Der Standardfehler ist ein Maß dafür, wie genau sich der arithmetische Mittelwert der Stichprobe an den tatsächlichen Mittelwert der Grundgesamtheit, den Erwartungswert, annähert (dazu gleich mehr) und wird auch *Stichprobenfehler* genannt. Der Standardfehler wird aus der Standardabweichung einer Messung und der Wurzel der Stichprobengröße berechnet. Da die Varianz in der Grundgesamtheit in der Regel unbekannt ist, wird der Standardfehler mit der Stichprobenvarianz geschätzt.

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}}$$

Manchmal wird der Standardfehler zur längeren Schreibweise umgeformt.

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Der Standardfehler wird umso kleiner (die Messung umso präziser), je kleiner die Varianz in der Grundgesamtheit und je größer der Stichprobenumfang ist.

Dies lässt sich mit einem simulierten Würfelexperiment verdeutlichen. Bei einem idealen, fairen Würfel kommt jede Augenzahl gleich oft vor. Der Erwartungswert eines sechsseitigen Würfels ist:

$$\frac{1}{6} \sum_{i=1}^{i=6} (x_i) = 3,5$$

Die Standardabweichung eines fairen, sechsseitigen Würfels beträgt:

$$\sqrt{\frac{1}{6} \sum_{i=1}^{i=6} (x_i - 3,5)^2} \approx 1,71$$

Da die Varianz in der Grundgesamtheit bekannt ist, hängt der Standardfehler des Mittelwerts eines fairen Würfels allein von der Stichprobengröße ab.

1.4.1 Experiment Verteilungskenngrößen

In einem simulierten Experiment würfeln 100 Personen jeweils 3, 10 und 50 Mal und bilden den Mittelwert der Augen. Weil ein fairer Würfel simuliert wird, kann der Standardfehler mit der Standardabweichung der Grundgesamtheit berechnet werden.

1.5 Ergebnisse

Würfe pro Person: 3
kleinster Mittelwert: 1.00
Stichprobenmittelwert: 3.57

Stichprobengröße: 300
größter Mittelwert: 6.00
Standardfehler: 0.10

Würfe pro Person: 10
kleinster Mittelwert: 1.70
Stichprobenmittelwert: 3.49

Stichprobengröße: 1000
größter Mittelwert: 4.60
Standardfehler: 0.05

Würfe pro Person: 50
kleinster Mittelwert: 2.98
Stichprobenmittelwert: 3.50

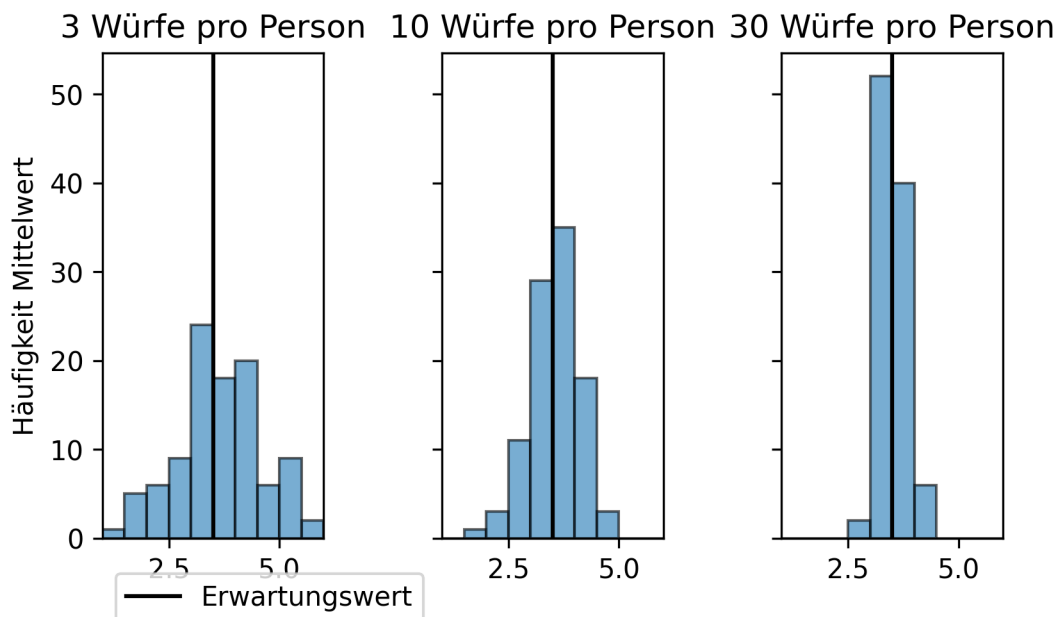
Stichprobengröße: 5000
größter Mittelwert: 4.36
Standardfehler: 0.02

Mit zunehmender Anzahl an Würfeln nähern sich Minimum und Maximum der individuellen Durchschnittswerte sowie der Stichprobenmittelwert dem Erwartungswert an.

Hinweis: Da das Skript dynamisch generiert wird, wurden die Zufallszahlen von einem festgelegten Startwert aus erzeugt.

1.6 grafische Darstellung

Die Häufigkeit der individuellen Mittelwerte ist in den folgenden Histogrammen dargestellt.



1.7 Code

Berechnung

```
personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen3.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen3.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen3.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen3.size ** (1/2) ):.2f}\n",
      sep = "")

# 10 Würfe
würfe = 10

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen10.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen10.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen10.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen10.size ** (1/2) ):.2f}\n",
      sep = "")

# 50 Würfe
würfe = 50

## Personen stehen in den Zeilen (axis = 1), Würfe in den Spalten (axis = 1)
```



```

augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 50)

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen50.mean(axis = 1).min():.2f}\t\t",
      f"größter Mittelwert: {augen50.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen50.mean():.2f}\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen50.size ** (1/2) ):.2f}\n",
      sep = "")

```

Darstellung

```

personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 3 * 100)

# 10 Würfe
würfe = 10
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 10 * 100)

# 50 Würfe
würfe = 50
augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 50 * 100)

# plotten
bins = 10

# 3 Würfe
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey = True)

ax1.hist(augen3.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax1.set_xlim(1, 6)
ax1.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black', label = 'Erwartungswert')
ax1.set_ylabel('mittleres Würfelergebnis')
ax1.set_ylabel('Häufigkeit Mittelwert')
ax1.set_title("3 Würfe pro Person")
ax1.legend(loc = 'lower left', bbox_to_anchor = (0, -0.2))

```

```

# 10 Würfe
ax2.hist(augen10.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax2.set_xlim(1, 6)
ax2.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax2.set_ylabel('mittleres Würfelergebnis')
ax2.set_title("10 Würfe pro Person")

# 30 Würfe
ax3.hist(augen50.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax3.set_xlim(1, 6)
ax3.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax3.set_ylabel('mittleres Würfelergebnis')
ax3.set_title("30 Würfe pro Person")

plt.tight_layout()
plt.show()

```

1.7.1 Aufgabe Verteilungskenngrößen

Im Datensatz `ToothGrowth.csv` ist eine Messreihe zur Länge zahnbildender Zellen bei Meerschweinchen gespeichert. Die Tiere erhielten Vitamin C direkt (VC) oder in Form von Orangensaft (OJ) in unterschiedlichen Dosen.

Code-Block 1.1

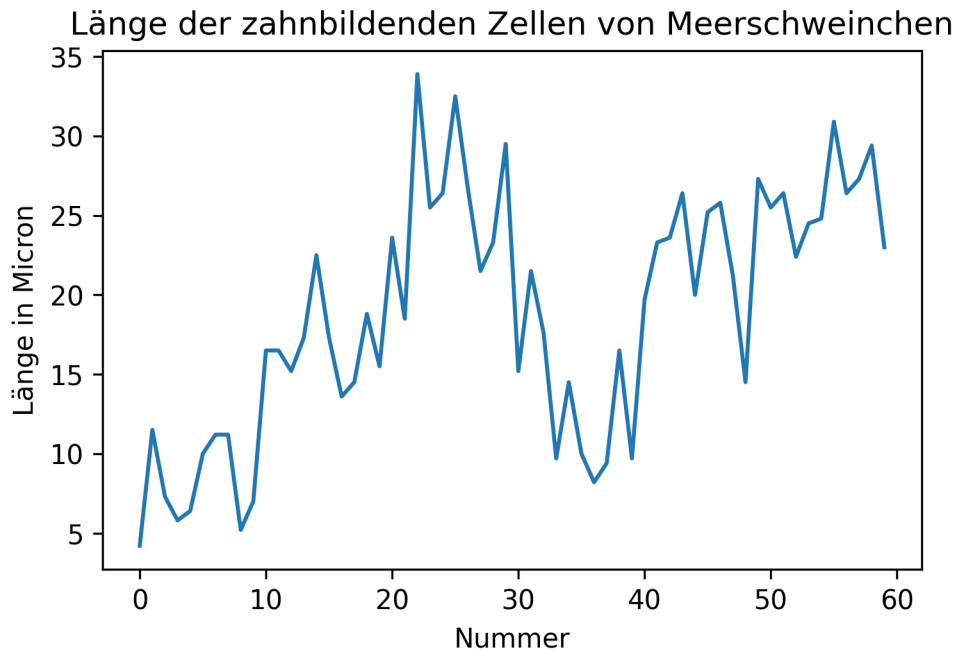
```

dateipfad = "01-daten/ToothGrowth.csv"
meerschweinchen = pd.read_csv(filepath_or_buffer = dateipfad, sep = ',', header = 0, \
    names = ['ID', 'len', 'supp', 'dose'], dtype = {'ID': 'int', 'len': 'float', 'dose': 'float'})

```

Crampton, E. W. 1947. „THE GROWTH OF THE ODONTOBLASTS OF THE INCISOR TOOTH AS A CRITERION OF THE VITAMIN C INTAKE OF THE GUINEA PIG“. The Journal of Nutrition 33 (5): 491–504. <https://doi.org/10.1093/jn/33.5.491>

Der Datensatz kann in R mit dem Befehl “`ToothGrowth`” aufgerufen werden.



Berechnen Sie den arithmetischen Mittelwert, die Varianz, die Standardabweichung und den Stichprobenfehler der Messreihe zur Zahnlänge (len). Verwenden Sie dazu die vorgestellten Formeln.

Das Ergebnis könnte so aussehen:

N: 60

arithmetisches Mittel: 18.81

Stichprobenfehler: 0.99

Stichprobenvarianz: 58.51

Standardabweichung: 7.65

💡 Tipp 1: Musterlösung Verteilungskenngrößen

```
def verteilungskennwerte(x, output = True):

    # Anzahl Messwerte bestimmen
    N = len(x)

    # arithmetisches Mittel bestimmen
    stichprobenmittelwert = sum(x) / N

    # Stichprobenvarianz bestimmen
    stichprobenvarianz = sum((x - stichprobenmittelwert) ** 2) / (N - 1)

    # Standardabweichung bestimmen
    standardabweichung = stichprobenvarianz ** (1/2)

    # Stichprobenfehler bestimmen
    stichprobenfehler = standardabweichung / (N ** (1/2))

    # Ausgabe
    if output: # output = True
        print(f"N: {N}\n",
              f"arithmetisches Mittel: {stichprobenmittelwert:.2f}\n",
              f"Stichprobenfehler: {stichprobenfehler:.2f}\n",
              f"Stichprobenvarianz: {stichprobenvarianz:.2f}\n",
              f"Standardabweichung: {standardabweichung:.2f}\n",
              sep = '')

    else: # output = False
        return N, stichprobenmittelwert, stichprobenfehler, stichprobenvarianz, standardabweichung

verteilungskennwerte(meerschweinchen['len'])
```

Die Module NumPy und Pandas verfügen über eigene Funktionen zur Berechnung der Varianz und der Standardabweichung (siehe folgendes Beispiel).

1.7.2 Varianz und Standardabweichung mit NumPy und Pandas

Die Varianz und Standardabweichung werden mit den Funktionen `np.var()` und `np.std()` bzw. den Methoden `pd.var()` und `pd.std()` berechnet. Der Parameter `ddof` (delta degrees of freedom) steuert, welcher Nenner zur Berechnung der Varianz verwendet wird in der Form N

- ddof. Während der Standardwert in NumPy 0 ist, berechnet Pandas mit dem Standardwert ddof=1 die Stichprobenvarianz.

```
print("Varianz:")
print(f"NumPy:\t{np.var(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].var():.2f}")

print("\nStandardabweichung:")
print(f"NumPy:\t{np.std(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].std():.2f}")
```

Varianz:

NumPy: 57.54

Pandas: 58.51

Standardabweichung:

NumPy: 7.59

Pandas: 7.65

2 Die Normalverteilung

Mit zunehmender Stichprobengröße wird eine immer bessere Schätzung des Erwartungswerts erreicht. Mathematisch liegt dieser Beobachtung der [zentrale Grenzwertsatz](#) zugrunde. So werden beim Würfeln mit mehreren Würfeln weit vom Erwartungswert entfernte Wurfresultate immer unwahrscheinlicher. Dies lässt sich bereits mit wenigen Würfeln zeigen (siehe Beispiel).

i Hinweis 1: Häufigkeitsverteilung von Würfelresultaten

Für einen Würfel gibt es 6 mögliche Resultate, für 2 Würfel $6 * 6$ mögliche Kombinationen, für 3 Würfel $6 * 6 * 6$ Kombinationen und so weiter. Weil viele Kombinationen wertgleich sind, kommen Wurfresultate in der Nähe des Erwartungswerts häufiger vor als beispielsweise ein Einserpasch.

2.1 ein Würfel

```
ein_würfel = []

for i in range(1, 7):
    ein_würfel.append(i)

ein_würfel = pd.Series(ein_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(ein_würfel.value_counts(), "\n")
print(f"Durchschnitt: {ein_würfel.mean():.1f}")

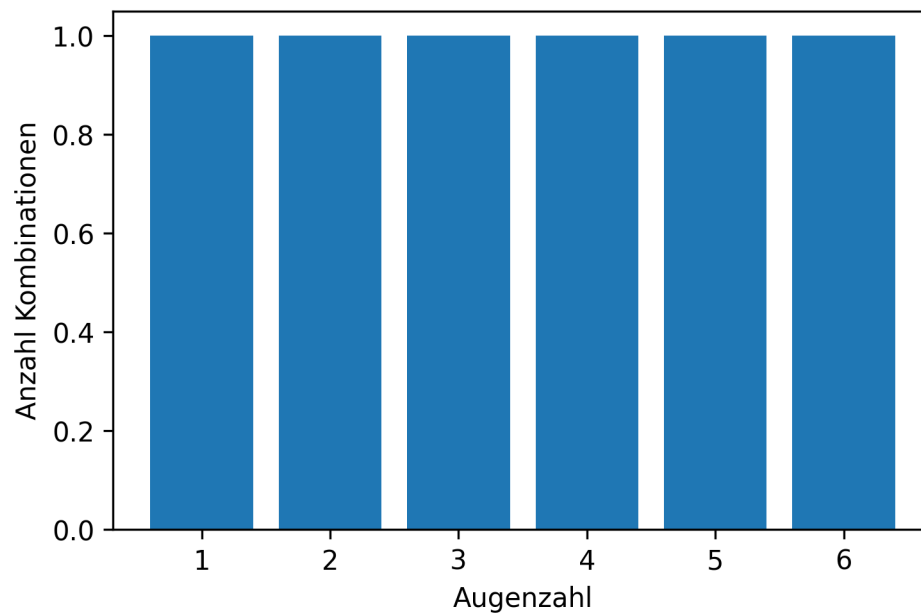
plt.bar(ein_würfel.unique(), ein_würfel.value_counts())
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.show()
```

Häufigkeitsverteilung der Augensumme:

1	1
2	1

```
3    1
4    1
5    1
6    1
Name: count, dtype: int64
```

Durchschnitt: 3.5



2.2 zwei Würfel

```

zwei_würfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j
        zwei_würfel.append(würfel_1 + würfel_2)

zwei_würfel = pd.Series(zwei_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(zwei_würfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {zwei_würfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {zwei_würfel.mean() / 2:.1f}")

plt.bar(zwei_würfel.unique(), zwei_würfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.grid()
plt.show()

```

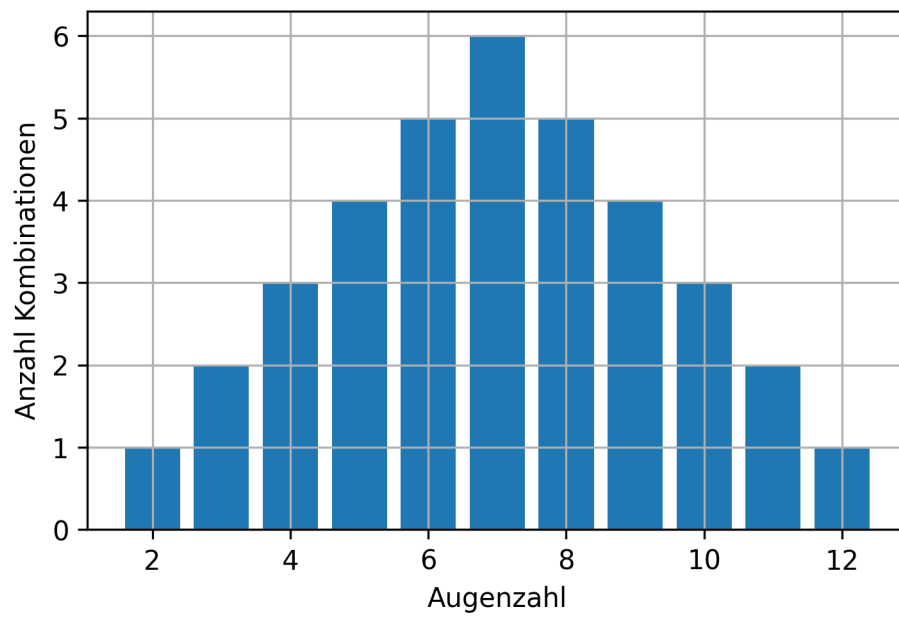
Häufigkeitsverteilung der Augensumme:

2	1
3	2
4	3
5	4
6	5
7	6
8	5
9	4
10	3
11	2
12	1

Name: count, dtype: int64

Durchschnitt: 7.0

Durchschnitt pro Würfel: 3.5



2.3 drei Würfel

```

dreiwürfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j

        for k in range (1, 7):
            würfel_3 = k
            dreiwürfel.append(würfel_1 + würfel_2 + würfel_3)

dreiwürfel = pd.Series(dreiwürfel)

print("Häufigkeitsverteilung der Augensumme:")
print(dreiwürfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {dreiwürfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {dreiwürfel.mean() / 3:.1f}")

plt.bar(dreiwürfel.unique(), dreiwürfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel ('Anzahl Kombinationen')
plt.grid()
plt.show()

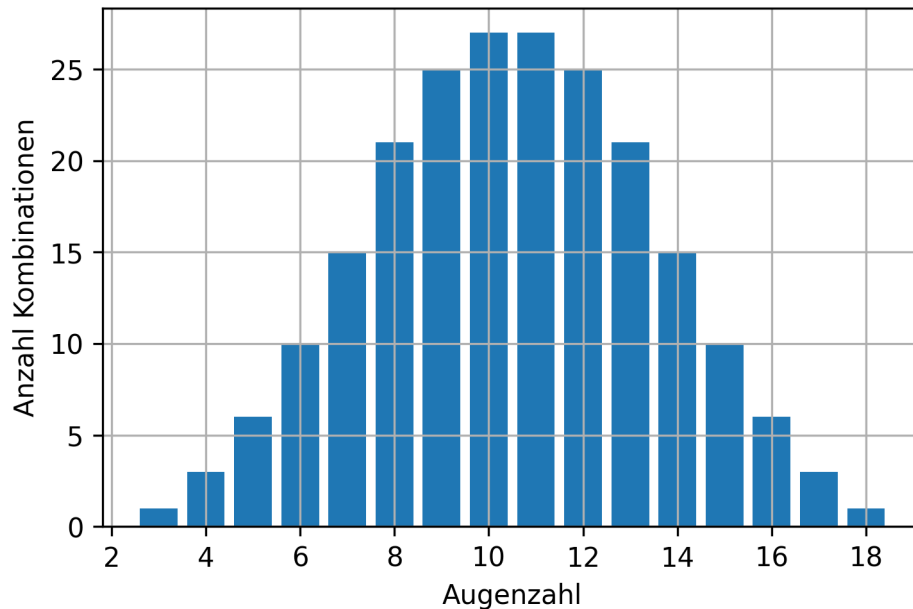
```

Häufigkeitsverteilung der Augensumme:

3	1
4	3
5	6
6	10
7	15
8	21
9	25
10	27
11	27
12	25
13	21
14	15
15	10
16	6
17	3

```
18      1
Name: count, dtype: int64

Durchschnitt: 10.5
Durchschnitt pro Würfel: 3.5
```



Die mit steigender Stichprobengröße zu beobachtende Annäherung von Messwerten an einen in der Grundgesamtheit geltenden Erwartungswert gilt auch, wenn der Erwartungswert und die Varianz in der Grundgesamtheit unbekannt sind. Mit zunehmender Stichprobengröße nähern sich die Messwerte der [Normalverteilung](#) an, die nach ihrem Entdecker Carl Friedrich Gauß auch als Gaußsche Glockenkurve bekannt ist.

Die für größere Stichproben zu beobachtende Annäherung der Verteilung von Messwerten an die Normalverteilung kann anhand des Gewichts von Pinguinen aus dem Datensatz `palmerpenguins` gezeigt werden.

palmerpenguins

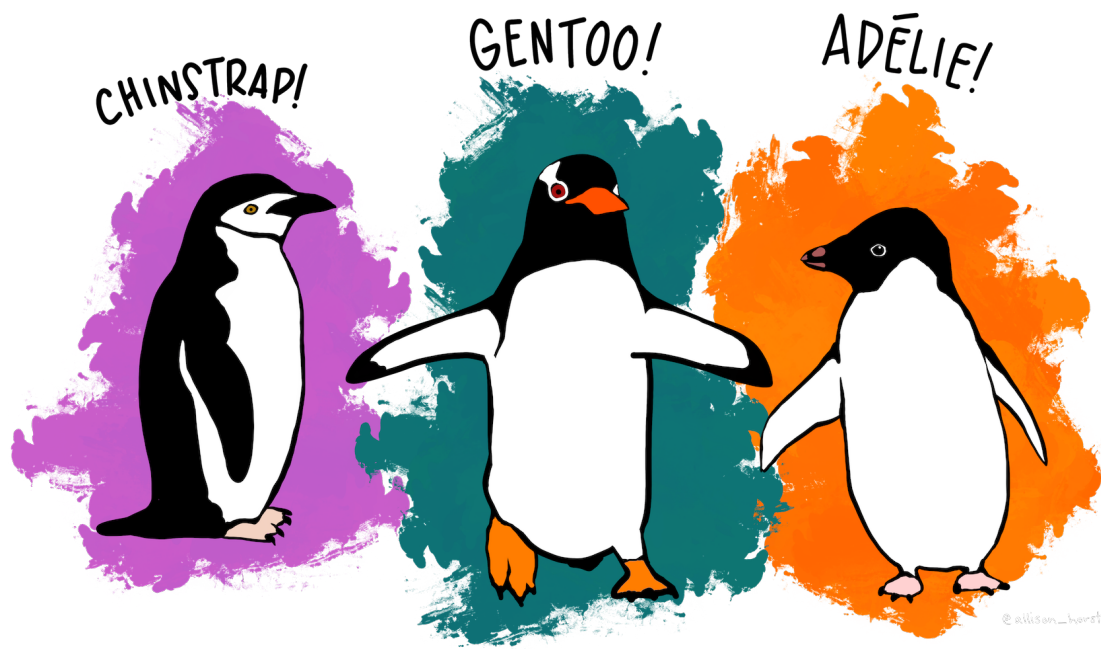


Abbildung 2.1: Pinguine des Palmer-Station-Datensatzes

Meet the Palmer penguins von @allison_horst steht unter der Lizenz [CC0-1.0](#) und ist auf [GitHub](#) abrufbar. 2020

Der Datensatz steht unter der Lizenz [CCO](#) und ist in R sowie auf [GitHub](#) verfügbar. 2020

```
# R Befehle, um den Datensatz zu laden
install.packages("palmerpenguins")
library(palmerpenguins)
```

Horst AM, Hill AP und Gorman KB. 2020. palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. <https://allisonhorst.github.io/palmerpenguins/>. doi: 10.5281/zenodo.3960218.

```
penguins = pd.read_csv(filepath_or_buffer = "01-daten/penguins.csv")

# Tiere mit unvollständigen Einträgen entfernen
penguins.drop(np.where(penguins.apply(pd.isna).any(axis = 1))[0], inplace = True)

print(penguins.info(), "\n");
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 333 entries, 0 to 343
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                333 non-null   object
1   island                 333 non-null   object
2   bill_length_mm         333 non-null   float64
3   bill_depth_mm          333 non-null   float64
4   flipper_length_mm      333 non-null   float64
5   body_mass_g            333 non-null   float64
6   sex                    333 non-null   object
7   year                   333 non-null   int64
dtypes: float64(4), int64(1), object(3)
memory usage: 23.4+ KB
None

```

Der Datensatz enthält Daten für drei Pinguinarten.

```
print(penguins.groupby(by = penguins['species']).size())
```

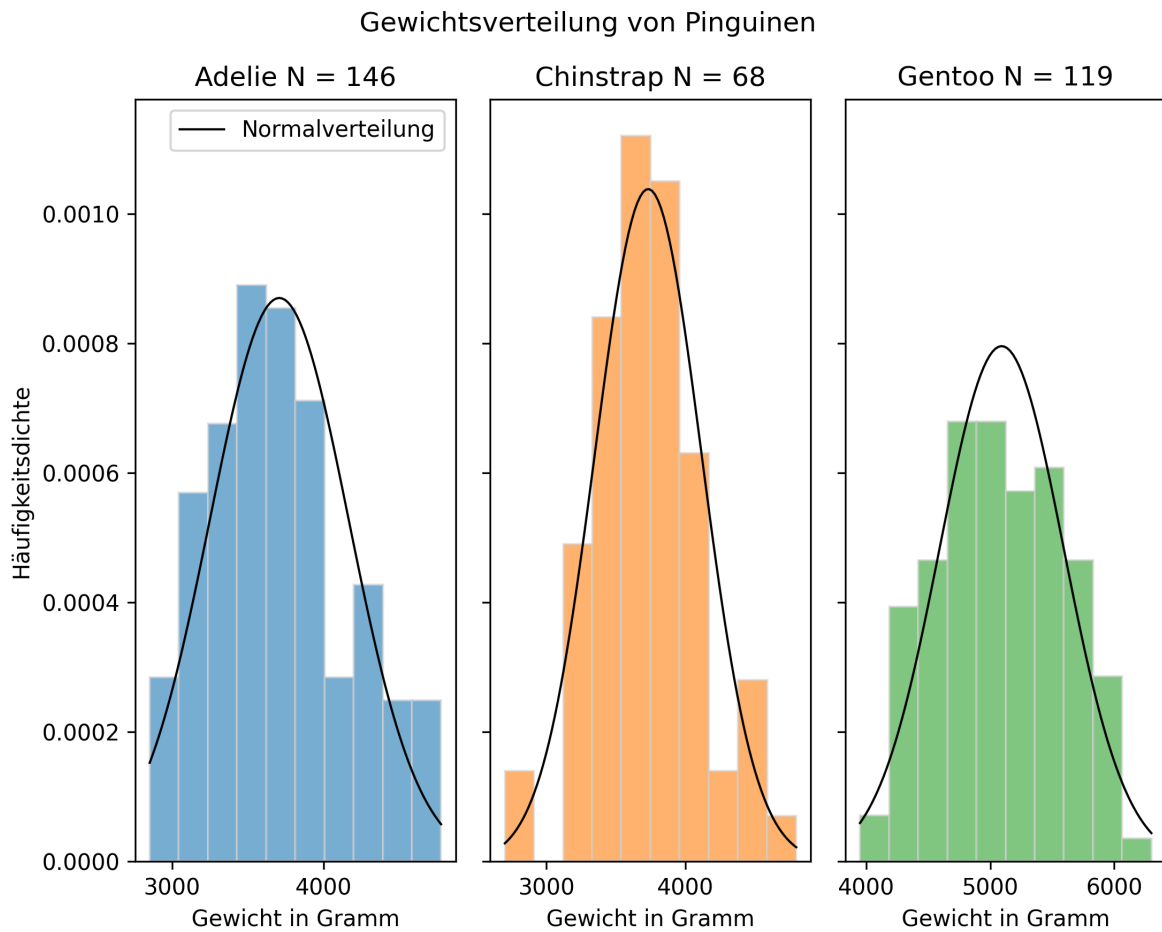
```

species
Adelie      146
Chinstrap   68
Gentoo     119
dtype: int64

```

Unter anderen wurde das Körpergewicht in Gramm gemessen, das in der Spalte 'body__mass_g' eingetragen ist. Die Gewichtsverteilung der drei Spezies wird jeweils mit einem Histogramm dargestellt. Außerdem werden für jede Spezies der Stichprobenmittelwert und die Stichprobenstandardabweichung bestimmt. Mit diesen Werten kann eine Normalverteilungskurve berechnet und in das Histogramm eingezeichnet werden (wie das geht, wird in Beispiel 3 gezeigt). So kann optisch geprüft werden, ob die empirische Verteilung der Werte in der Stichprobe einer Normalverteilung mit den selben Werten für Mittelwert und Standardabweichung entspricht.

2.4 Grafik



2.5 Code

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von Pinguinen')

# Adelie
species = 'Adelie'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
```

```

ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(- (x_values - s

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 'Normalverteilung')
ax1.legend()

# Chinstrap
species = 'Chinstrap'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(- (x_values - s

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve

```

```

stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(-(x_values - stichprobenmittelwert)**2 / (2 * stichprobenstandardabweichung**2))

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

Die Normalverteilung ist eine Dichtekurve, an die sich der Verlauf eines Histogramms mit einer gegen unendlich gehenden Anzahl von Messwerten und einer gegen Null gehenden Klassenbreite annähert.

! Wichtig 2: Histogramm

Das Histogramm ist eine grafische Darstellung der Häufigkeitsverteilung kardinal skalierten Merkmale. Die Daten werden in Klassen, die eine konstante oder variable Breite haben können, eingeteilt. Es werden direkt nebeneinanderliegende Rechtecke von der Breite der jeweiligen Klasse gezeichnet, deren Flächeninhalte die (relativen oder absoluten) Klassenhäufigkeiten darstellen. Die Höhe jedes Rechtecks stellt dann die (relative oder absolute) Häufigkeitsdichte dar, also die (relative oder absolute) Häufigkeit dividiert durch die Breite der entsprechenden Klasse.

Die Dichtefunktion der Normalverteilung beschreibt, welcher Anteil der Werte innerhalb eines bestimmten Wertebereichs liegt. Bei der Berechnung der relativen Häufigkeiten in Beispiel 2 haben wir gesehen, dass die Summe der relativen Häufigkeiten 1 ist. Dies entspricht der Fläche unterhalb der Dichtekurve.

Die Dichtefunktion der Normalverteilung ist definiert als:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Die Form der Normalverteilung ergibt sich aus dem Faktor $e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$ der Funktionsgleichung. Das Maximum der Funktion liegt am Punkt $x = \mu$. Von dort fällt sie symmetrisch ab und nähert sich der x-Achse an. Der Abfall der Funktion erfolgt umso schneller, je kleiner σ ist. Die Wendepunkte der Kurve liegen jeweils eine Standardabweichung vom Mittelwert entfernt.

Eine Normalverteilung mit dem Mittelwert $\mu = 0$ und einer Standardabweichung $\sigma = 1$ heißt Standardnormalverteilung.

2.12 Normalverteilung anpassen

Um die Verteilung in einem Datensatz durch eine Normalverteilung anzunähern, werden dessen Mittelwert und Standardabweichung in die Funktionsgleichung der Normalverteilung eingesetzt. Mit Python können die Berechnungen direkt vorgenommen werden. In der Handhabung einfacher sind die vom Paket SciPy bereitgestellten Funktionen, die im nächsten Abschnitt ausführlicher vorgestellt werden. Das folgende Beispiel zeigt die Berechnung und Visualisierung mit Python und mit SciPy.

Hinweis 3: Dichtekurven berechnen und darstellen

Betrachten wir die Verteilungskennwerte der Gruppe der Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten.

```
print(verteilungskennwerte(dose2), "\n");

dose2_mean = verteilungskennwerte(dose2, output = False)[1]
dose2_std = verteilungskennwerte(dose2, output = False)[4]

print("Exakter Mittelwert:", dose2_mean)
print("Exakte Standardabweichung:", dose2_std)
```

```
N: 20
arithmetisches Mittel: 26.10
```

Stichprobenfehler: 0.84
Stichprobenvarianz: 14.24
Standardabweichung: 3.77
None

Exakter Mittelwert: 26.1
Exakte Standardabweichung: 3.7741503052098744

Wenn wir die Standardabweichung und das arithmetische Mittel in die Normalverteilungsfunktion einsetzen, erhalten wir:

$$f(x) = \frac{1}{3.7742\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

$$f(x) = 0.1057 \cdot e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

In Python können die Berechnungen umgesetzt und grafisch dargestellt werden:

```
# Histogram der Häufigkeitsdichte zeichnen
plt.hist(dose2, bins = 7, density = True, edgecolor = 'black', alpha = 0.6);
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

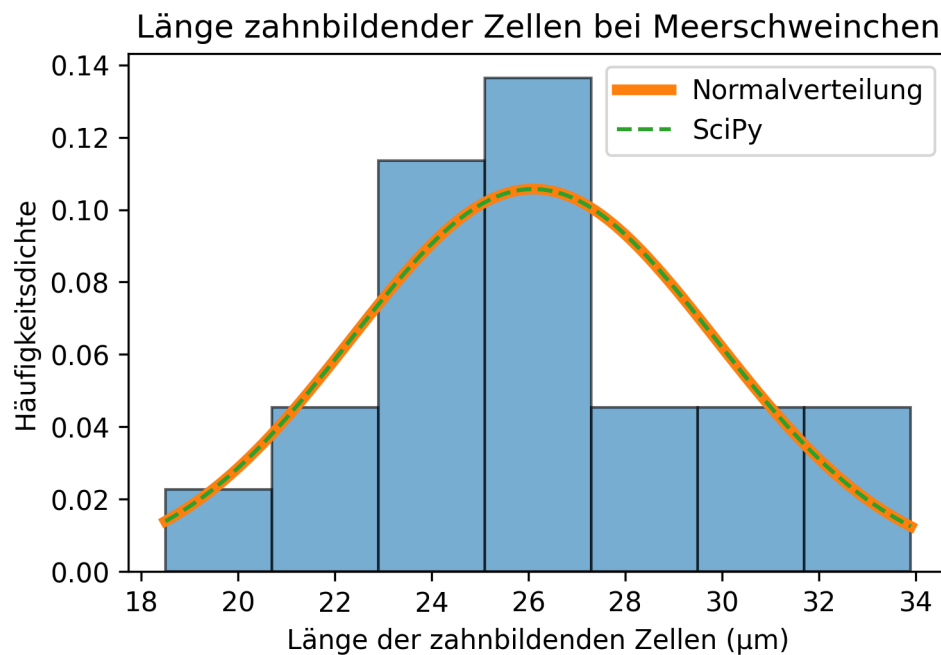
# Normalverteilung berechnen.
hist, bin_edges = np.histogram(dose2, bins = 7)

x_values = np.linspace(min(bin_edges), max(bin_edges), 100)

## Normalverteilungsfunktion mit Python berechnen
y_values = 1 / (dose2_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - dose2_mean) ** 2 /
plt.plot(x_values, y_values, label = 'Normalverteilung', lw = 4)

## scipy
y_values_scipy = scipy.stats.norm.pdf(x_values, loc = dose2_mean, scale = dose2_std)
plt.plot(x_values, y_values_scipy, label = 'SciPy', linestyle = 'dashed')

plt.legend()
plt.show()
```



Die Verteilung der Länge zahnbildender Zellen bei Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten, könnte einer Normalverteilung entsprechen. Aufgrund der geringen Stichprobengröße ist dies aber schwer zu beurteilen.

Quelle: Skript MB S. 51-54

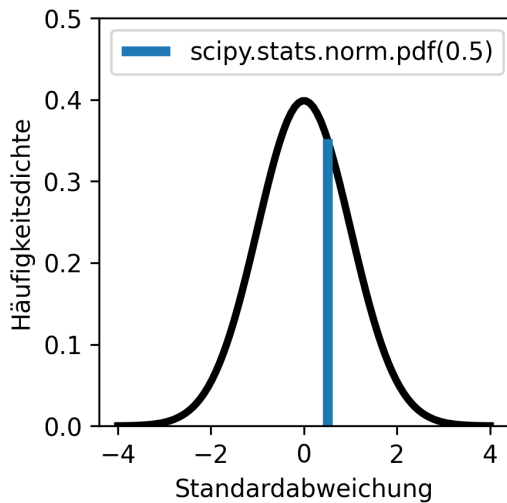
2.13 Das Paket SciPy

Funktionen zur Berechnung von Dichtekurven können über Paket SciPy importiert werden. Das Modul stats (statistical functions) umfasst zahlreiche Funktionen zum Testen von Hypothesen. Funktionen für die Normalverteilung werden wie folgt aufgerufen:

```
import scipy
print("Häufigkeitsdichte der Normalverteilung bei x = 0:", scipy.stats.norm.pdf(0), "\n")
```

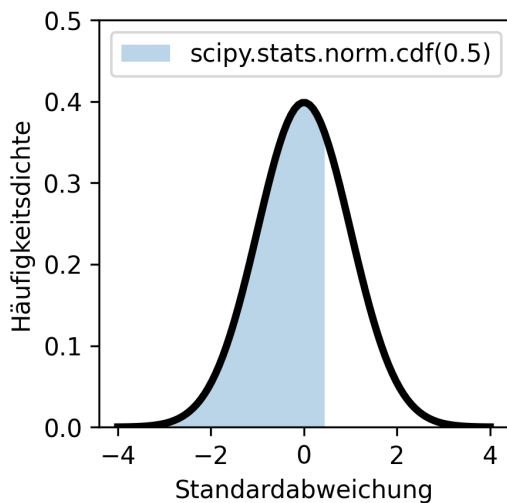
Häufigkeitsdichte der Normalverteilung bei x = 0: 0.3989422804014327

Für die Normalverteilung sind vier Funktionen relevant:



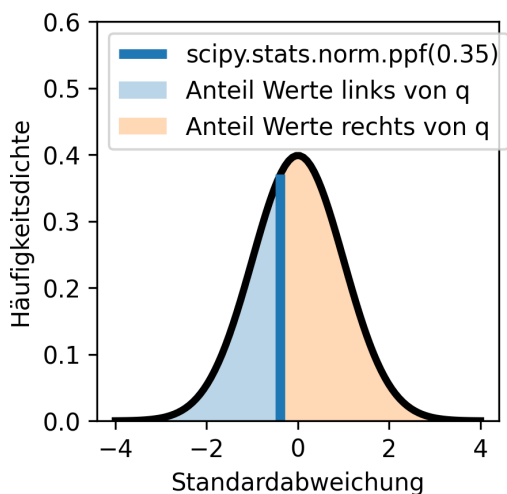
Beschreibung

Die Funktion `scipy.stats.norm.pdf(x)` berechnet die Dichte der Normalverteilung am Punkt x (PDF = probability density function). x kann auch ein array sein - so wurde die linksstehende Kurve mit dem Befehl `scipy.stats.norm.pdf(np.linspace(-4, 4, 100))` berechnet.



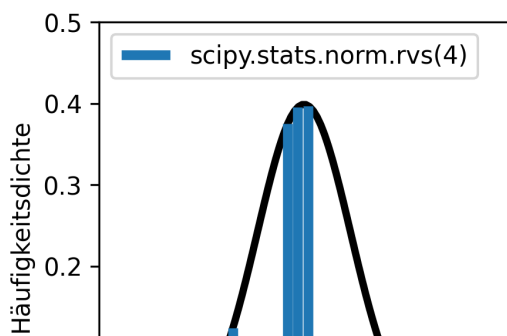
Beschreibung

Die Funktion `scipy.stats.norm.cdf(x)` berechnet den Anteil der Werte links von x (CDF = cumulative density function).



Beschreibung

Die Funktion `scipy.stats.norm.ppf(q)` ist die Quantilfunktion der Normalverteilung und die Umkehrfunktion der kumulativen Häufigkeitsdichtefunktion (CDF). Die Funktion berechnet für $0 \leq q \leq 1$ den Wert x , links von dem der Anteil q aller Werte liegt und rechts von dem der Anteil $1-q$ liegt (PPF = percentile point function).



Beschreibung

Die Funktion `scipy.stats.norm.rvs(size)` zieht `size` Zufallszahlen aus der Normalverteilung.

Hinweis: Die Zufallszahlen werden im Skript dynamisch gezogen.

Mit den Parametern `loc = mittelwert` und `scale = standardabweichung` kann die Form der Normalverteilung angepasst werden. Standardmäßig wird die Standardnormalverteilung mit `loc = 0` und `scale = 1` berechnet. Die Parameter der Funktionen können Einzelwerte (Skalare) oder auch Arrays bzw. Listen sein.

2.14 Aufgaben Normalverteilung

Möglicherweise haben Sie schon einmal von Mensa International gehört, einer Vereinigung für Hochbegabte. Wer Mitglied in dieser Vereinigung werden möchte, soll einen höheren Intelligenzquotienten (IQ) haben als 98 % der Bevölkerung seines/ihrer Herkunftslandes ([Wikipedia](#)).

1. Wenn der durchschnittliche IQ 100 und die Standardabweichung 15 beträgt, welchen IQ müssten Sie haben, um bei Mensa International aufgenommen zu werden?
2. Mensa International ist nicht die einzige Organisation ihrer Art, andere Organisationen haben sogar noch strengere Kriterien. Welcher IQ wird benötigt, um hier Mitglied zu werden?
 - Intertel (Kriterium: IQ aus dem höchsten 1 %)
 - Triple Nine Society (Kriterium: IQ aus dem höchsten 0,1 %)
 - Prometheus Society (Kriterium: IQ aus dem höchsten 0,003 %)
3. Der IQ ist nicht mit angeborener Intelligenz gleichzusetzen und auch abhängig davon, wie viel Gelegenheit man zum Gehirntraining hatte, etwa durch den Schulbesuch. Der niedrigste durchschnittliche IQ wurde mit 71 [im Land Niger](#) gemessen. Angenommen Sie hätten einen IQ von 100. Würden Sie in Niger das Kriterium der Mensa International erfüllen?

Musterlösung Normalverteilung

Aufgabe 1: Einen IQ von mehr als ...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.98))
```

130.80623365947733

Aufgabe 2: Sie benötigen einen IQ von mindestens...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.99))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.999))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 1 - (0.003 / 100)))
```

134.8952181106126

```
146.3534845925172
160.19216216677682
```

Aufgabe 3: Nicht ganz.

```
print(scipy.stats.norm.cdf(loc = 71, scale = 15, x = 100))
```

```
0.9734024259789904
```

Übrigens: Wie [der Spiegel berichtet](#), schneiden Studierende mit mittelmäßigem Intelligenzquotienten ebenso erfolgreich ab wie Hochbegabte, vorausgesetzt sie sind neugierig genug und arbeiten gewissenhaft.

2.15 Konfidenzintervalle

Die Grundidee in der Statistik ist, dass von Stichprobenwerten auf den tatsächlichen Wert in der Grundgesamtheit geschlossen werden kann. Die Überlegung ist wie folgt:

1. Wenn eine Stichprobe aus einer Grundgesamtheit gezogen wird, dann streuen die Stichprobenwerte normalverteilt um den Mittelwert der Grundgesamtheit. Bei einer Normalverteilung liegen
 - 68,27 % aller Werte im Intervall $\pm 1 s$,
 - 95,45 % aller Werte im Intervall $\pm 2 s$ und
 - 99,73 % aller Werte im Intervall $\pm 3 s$.
2. Mit der gleichen Wahrscheinlichkeitsverteilung liegt der unbekannte Mittelwert der Grundgesamtheit um einen zufälligen Wert aus der Stichprobe.
3. Der Erwartungswert kann mit einer gewissen Wahrscheinlichkeit aus dem Standardfehler des Mittelwerts einer Stichprobe geschätzt werden. Man wählt dazu ein Konfidenzniveau, also eine Vertrauenswahrscheinlichkeit, dass der Erwartungswert tatsächlich im Bereich der Schätzung liegt. Der umgekehrte Fall, dass der Erwartungswert nicht im Bereich der Schätzung liegt, wird Signifikanz- oder Alphaniveau genannt und mit dem griechischen Buchstaben α (alpha) gekennzeichnet. α liegt im Bereich 0 - 1, das Konfidenzniveau ist $1 - \alpha$ (siehe: [Fehler 1. und 2. Art](#)).
 - der Erwartungswert liegt in 68,27 % aller Fälle im Intervall $\pm 1 \frac{s}{\sqrt{n}}$,
 - der Erwartungswert liegt in 95,45 % aller Fälle im Intervall $\pm 2 \frac{s}{\sqrt{n}}$ und
 - der Erwartungswert liegt in 99,73 % aller Fälle im Intervall $\pm 3 \frac{s}{\sqrt{n}}$.

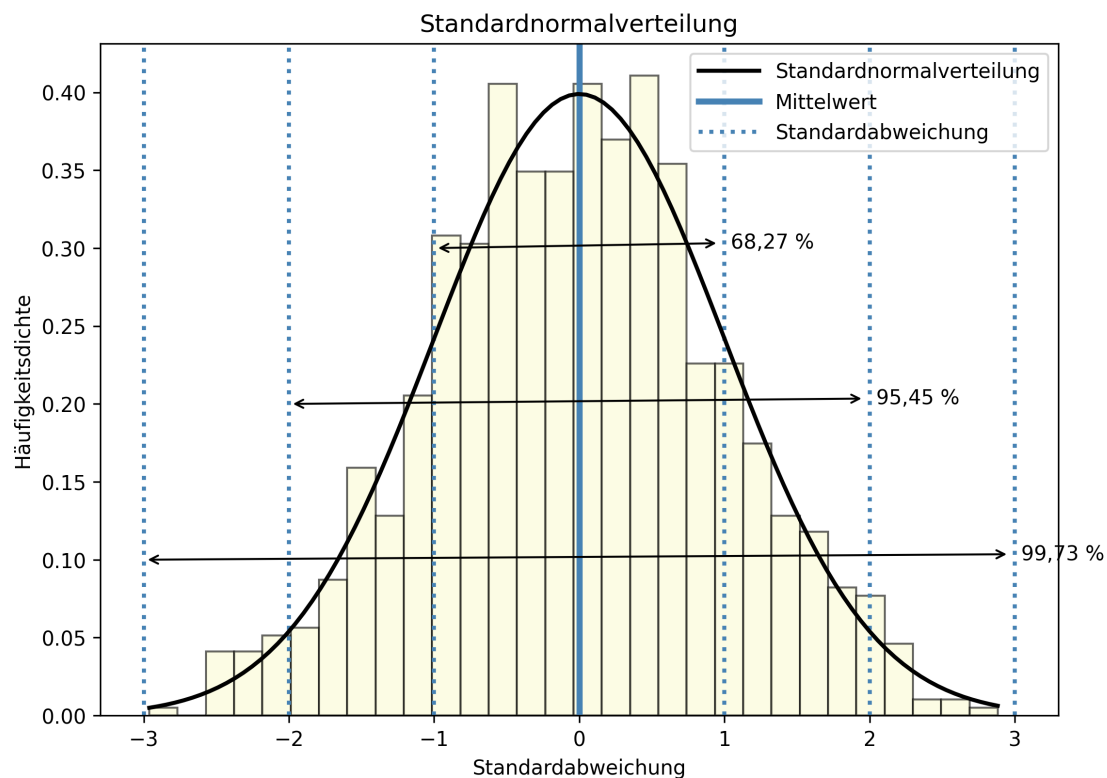
Häufig wird das Alphaniveau $\alpha = 0.05$ bzw. das Konfidenzintervall 95 % gewählt, was $\pm 1.96 \frac{s}{\sqrt{n}}$ entspricht. Dies gilt aber nur für große Stichproben. Für kleine Stichprobengrößen folgen die Stichprobenmittelwerte der t-Verteilung, die im nächsten Abschnitt vorgestellt wird.

hier könnte / müsste man noch einseitige und zweiseitige Hypothesentests und den Begriff “Alpha-Halbe” einführen. Das ließe sich auch gut grafisch mit nur nach rechts gehenden und beidseitigen Pfeilen darstellen.

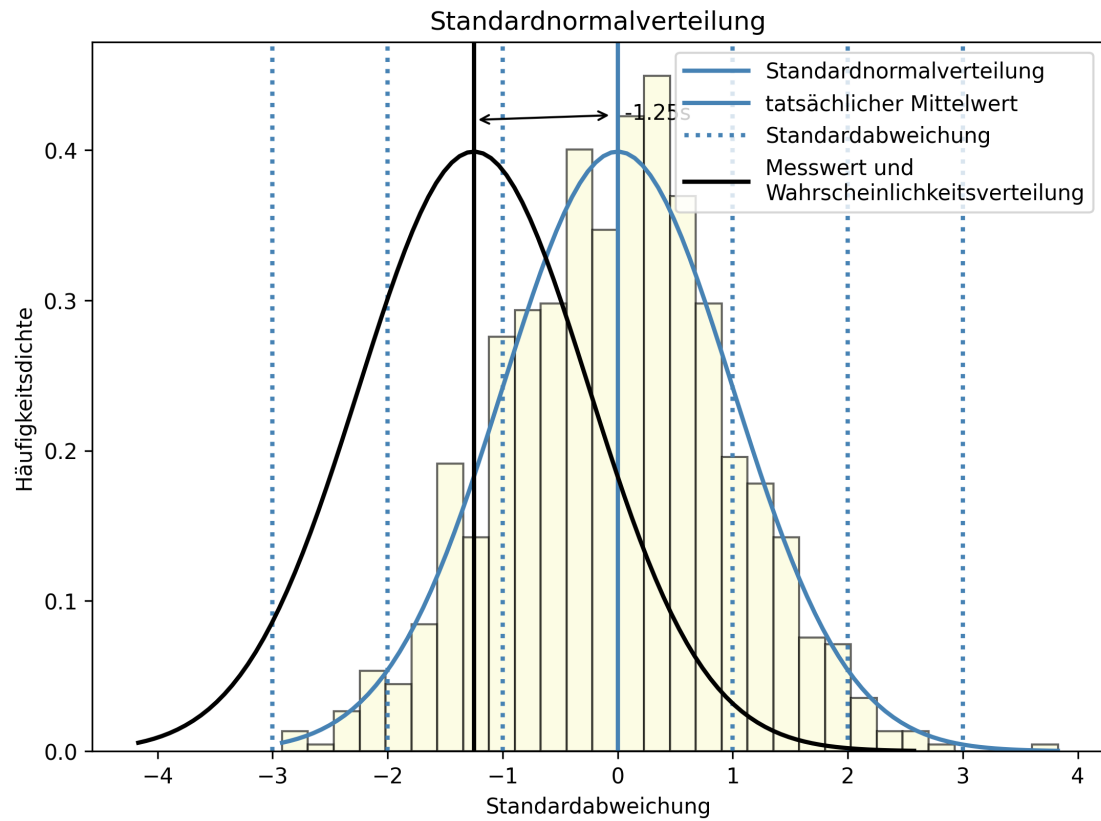
Im folgenden Beispiel wird die Idee, dass mit einer gewissen Wahrscheinlichkeit vom Stichprobenmittelwert auf den Mittelwert der Grundgesamtheit (Erwartungswert) geschlossen werden kann, noch einmal grafisch dargestellt.

Hinweis 4: Prinzip der schließenden Statistik

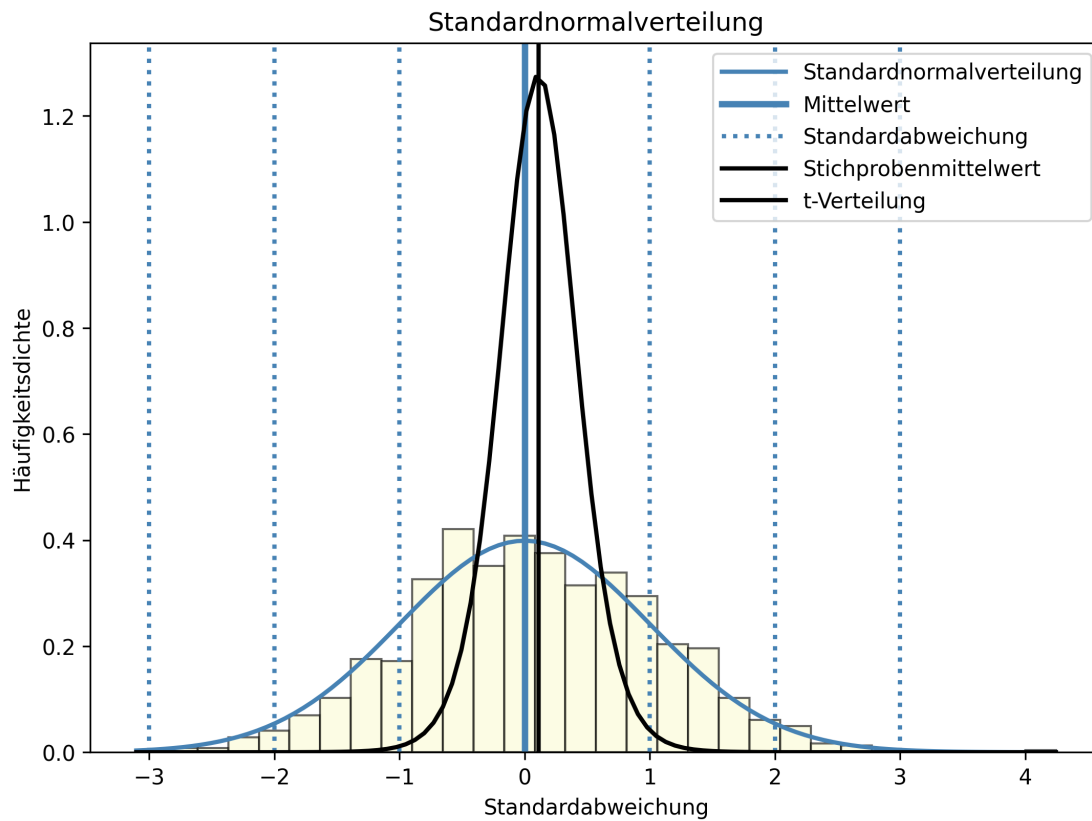
2.16 Standardnormalverteilung



2.17 Einzelner Messwert



2.18 Stichprobe $N = 12$



Ein Stichprobenmittelwert streut inform einer Normalverteilungskurve mit $\sigma = \text{stichprobenfehler}$. Diese Dichtekurve ist erheblich schmäler und höher als die Normalverteilungskurve eines einzelnen Messwerts. Dies liegt an der geringen Standardabweichung in der Stichprobe von ~ 0.2 , was die Kurve staucht.

2.19 Code

Code für das Panel Stichprobe $N = 12$

```

import numpy as np
import matplotlib.pyplot as plt
import scipy

# Parameter der Standardnormalverteilung
mu, sigma = 0, 1 # Mittelwert und Standardabweichung

# Daten generieren
seed = 4
np.random.seed(seed = seed)
data = np.random.default_rng().normal(mu, sigma, 1000)

# Grafik
plt.figure(figsize = (8.5, 6))

# Histogramm plotten
array, bins, patches = plt.hist(data, bins = 30, density = True, alpha = 0.6, color = 'lightblue')

# Mittelwert einzeichnen
mean_line = plt.axvline(mu, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# positive und negative Standardabweichungen einzeichnen
pos_std_lines = [plt.axvline(mu + i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]
neg_std_lines = [plt.axvline(mu - i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]

# Normalverteilungskurve
x_values = np.linspace(min(bins), max(bins), 100)
y_values = 1 / (sigma * np.sqrt(2 * np.pi)) * np.exp(- (x_values - mu) ** 2 / (2 * sigma ** 2))
normal_dist_curve = plt.plot(x_values, y_values, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# Stichprobe
N = 12
np.random.seed(seed = 4)
stichprobe = np.random.default_rng().normal(mu, sigma, N)

stichprobenstandardabweichung = stichprobe.std(ddof = 1)
stichprobenmittelwert = stichprobe.mean()
standardfehler = stichprobenstandardabweichung / np.sqrt(len(stichprobe))

# Histogramm berechnen
# hist, bins = np.histogram(stichprobe, bins = 30, density = True)

# Standardfehlerkurve Stichprobe
# x_values = np.linspace(min(bins), max(bins), 100)
x = np.linspace(stichprobenmittelwert - 4 * stichprobenstandardabweichung, stichprobenmittelwert + 4 * stichprobenstandardabweichung, 100)
y_values = scipy.stats.t.pdf(x = x_values, df = N - 1, loc = stichprobenmittelwert, scale = standardfehler)

# Stichprobenmittelwert einzeichnen
mean_stichprobe = plt.axvline(stichprobenmittelwert, color = 'black', linestyle = 'solid', linewidth = 3)

# Verteilungskurve einzeichnen
stichprobe_dist_curve = plt.plot(x_values, y_values, color = 'black', linestyle = 'solid', linewidth = 3)

```

2.20 Die t-Verteilung

[Wikipedia](#)

[Anzahl der Freiheitsgrade](#)

! Wichtig 3: Anzahl Freiheitsgrade

“Die Anzahl unabhängiger Information, die in die Schätzung eines Parameters einfließen, wird als Anzahl der Freiheitsgrade bezeichnet. Im Allgemeinen sind die Freiheitsgrade einer Schätzung eines Parameters gleich der Anzahl unabhängiger Einzelinformationen, die in die Schätzung einfließen, abzüglich der Anzahl der zu schätzenden Parameter, die als Zwischenschritte bei der Schätzung des Parameters selbst verwendet werden. Beispielsweise fließen n Werte in die Berechnung der Stichprobenvarianz ein. Dennoch lautet die Anzahl der Freiheitsgrade $n - 1$, da als Zwischenschritt der Mittelwert geschätzt wird und somit ein Freiheitsgrad verloren geht.”

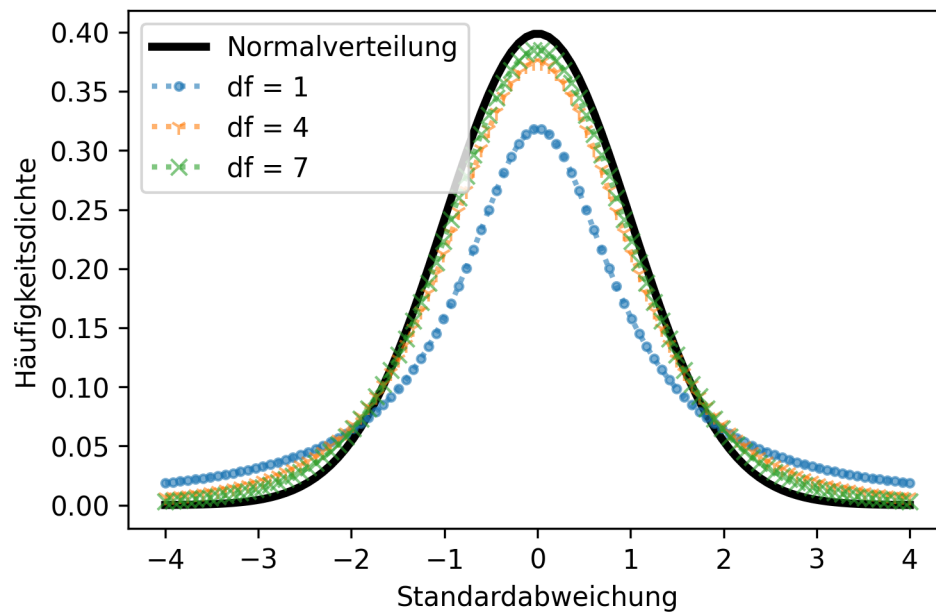
Anzahl der Freiheitsgrade (Statistik). von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia](#). 2025

-
- [Gammafunktion](#)

-
-
-
-
-

2.21 Grafik

Das Argument df der t-Verteilung



2.22 Code

```
x_values = np.linspace(-4, 4, 100)

# Normalverteilung
y_values = scipy.stats.norm.pdf(x_values)
plt.plot(x_values, y_values, color = 'black', lw = 3, label = 'Normalverteilung')
# plt.ylim(bottom = 0, top = 0.5)

# t-Verteilungen
marker = [".", "1", "x"]

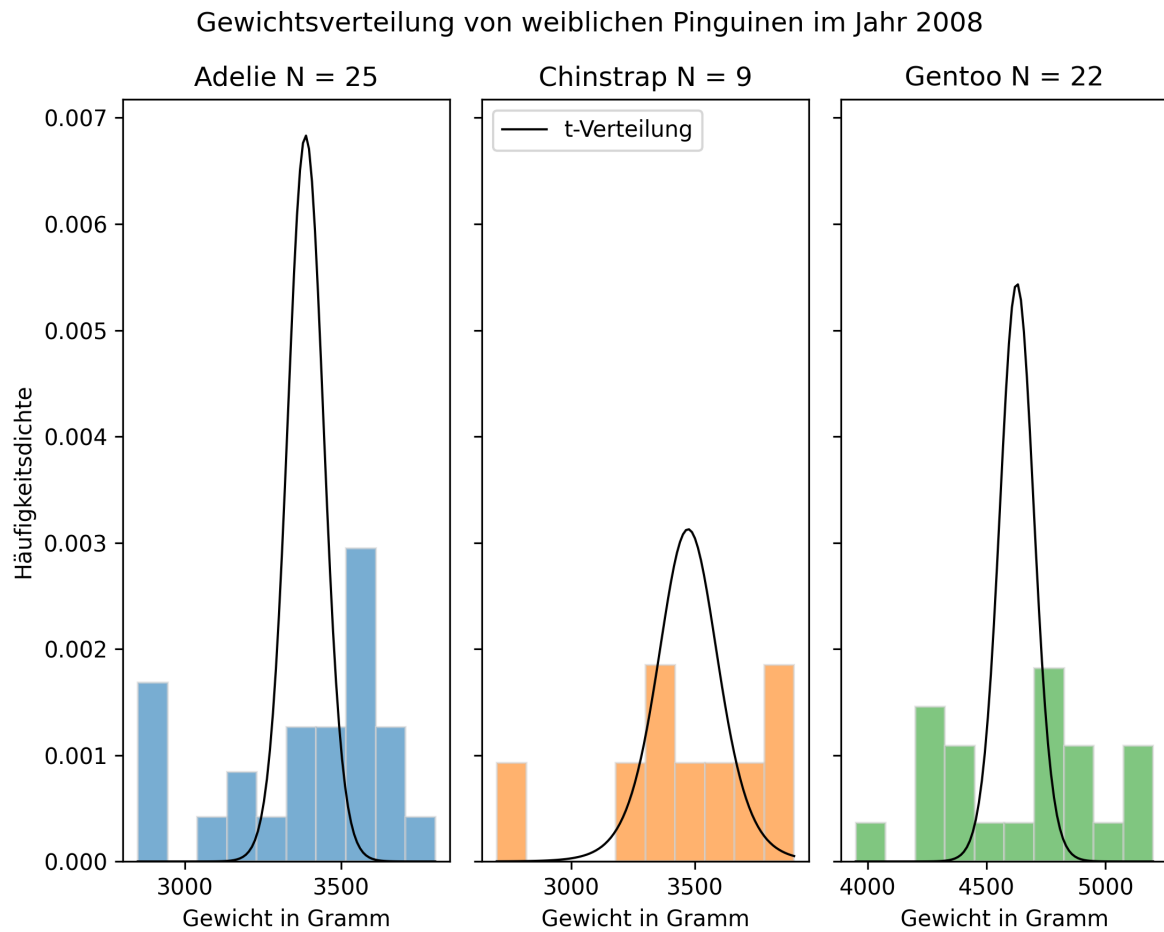
[plt.plot(x_values, scipy.stats.t.pdf(x_values, df = (i + (i - 1) * 2)), linestyle = 'dotted',
         marker = marker[i % 3], label = f't-Verteilung, df = {i + (i - 1) * 2}') for i in range(1, 10)]

plt.suptitle('Das Argument df der t-Verteilung')
plt.xlabel('Standardabweichung')
plt.ylabel('Häufigkeitsdichte')
plt.legend(loc = 'upper left')
plt.show()
```

- -
 -
-
-

```
print(penguins.groupby(by = [penguins['species'], penguins['sex'], penguins['year']]).size())
```


2.23 Grafik



2.24 Code

```
year = 2008
sex = 'female'
species = 'Adelie'

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von weiblichen Pinguinen im Jahr 2008')

# Adelie
data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
```



```

stichprobengröße = data.size

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 't-Verteilung')
ax1.legend(loc = 'upper left')

# Chinstrap
species = 'Chinstrap'

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'

```

```

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

2.25 Aufgabe Konfidenzintervalle

- 1.
- 2.

💡 Tipp 2: Tipp und Musterlösung

Folgende Schritte helfen Ihnen bei der Lösung:

1. Bestimmen Sie den Stichprobenmittelwert $\bar{x}$.
2. Bestimmen Sie die Stichprobenstandardabweichung s , die Stichprobengröße N und den Standardfehler $\frac{s}{\sqrt{N}}$.
3. Bestimmen Sie die z- oder t-Werte der Normal- bzw. t-Verteilung für das gewählte Konfidenzniveau - für einen zweiseitigen Hypothesentest $\frac{\alpha}{2}$ und $1 - \frac{\alpha}{2}$.
4. Berechnen Sie das Konfidenzintervall $\bar{x} \pm t_{\alpha/2} \frac{s}{\sqrt{n}}$.

💡 Musterlösung

Alphaniveau definieren und Pinguine auswählen

```
alpha = 1 - 0.9
data = penguins[(penguins['sex'] == sex) & (penguins['year'] == year)]
```

1. Stichprobenmittelwerte bestimmen.

```
penguin_means = data['body_mass_g'].groupby(by = data['species']).mean()
print(penguin_means)
```

```
species
Adelie      3386.000000
Chinstrap   3472.222222
Gentoo      4627.272727
Name: body_mass_g, dtype: float64
```

2. Stichprobenstandardabweichung, Stichprobengröße und Standardfehler bestimmen.

```
penguin_stds = data['body_mass_g'].groupby(by = data['species']).std(ddof = 1)
penguin_sizes = data['body_mass_g'].groupby(by = data['species']).size()
penguin_stderrors = penguin_stds / np.sqrt(penguin_sizes)

print("Stichprobenstandardabweichungen:\n", penguin_stds)
print("\nStichprobengrößen:\n", penguin_sizes)
print("\nStandardfehler:\n", penguin_stderrors)
```

```
Stichprobenstandardabweichungen:
species
Adelie      288.862712
Chinstrap   370.903551
Gentoo      339.722321
Name: body_mass_g, dtype: float64
```

51

```
Stichprobengrößen:
species
Adelie      25
Chinstrap    9
Gentoo      22
Name: body_mass_g, dtype: int64
```

3 Lineare Parameterschätzung

-
-

3.1 Messreihe Hooke'sches Gesetz

Hooke'sche Gesetz

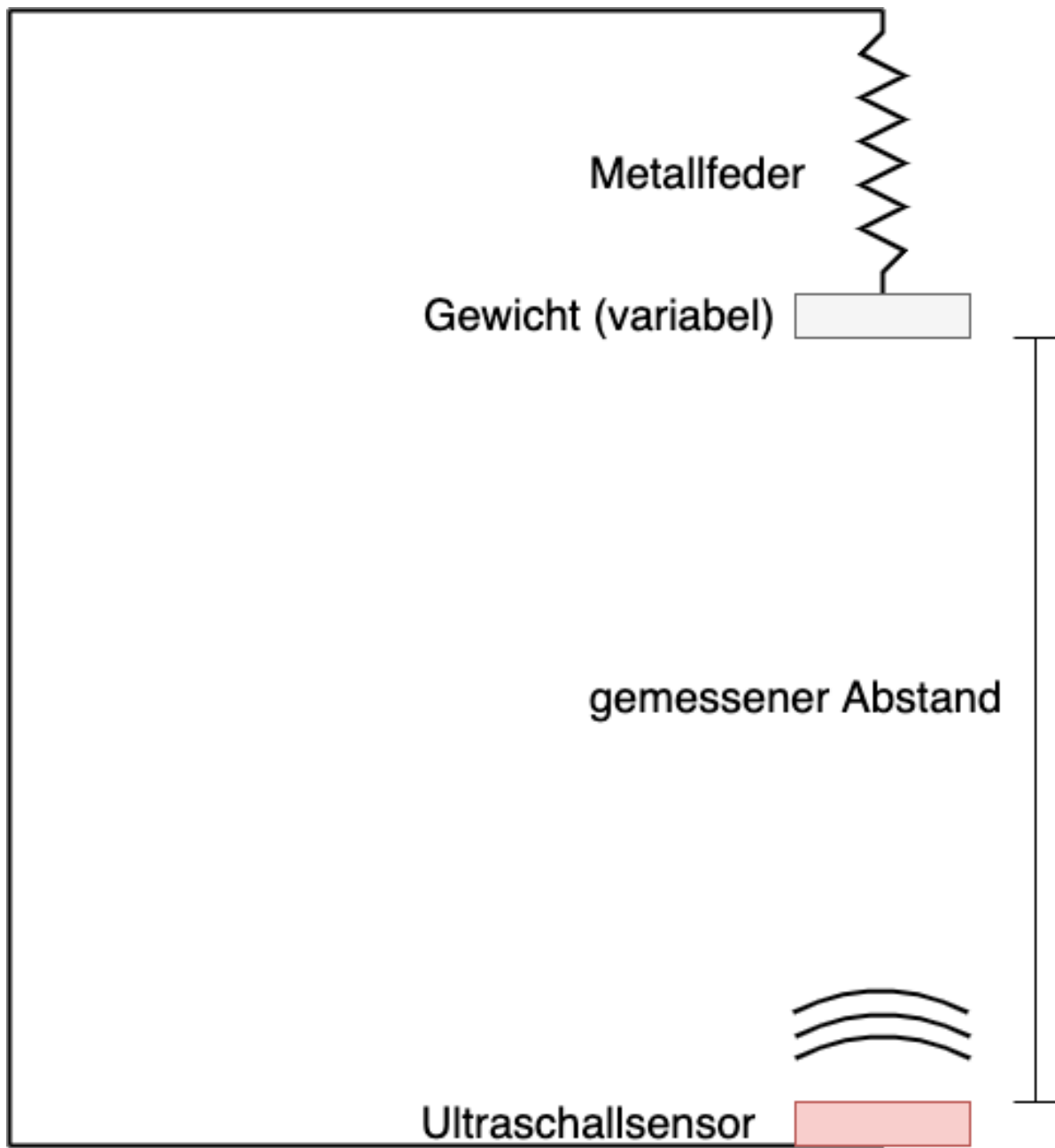


Abbildung 3.1: Versuchsaufbau

```
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
```

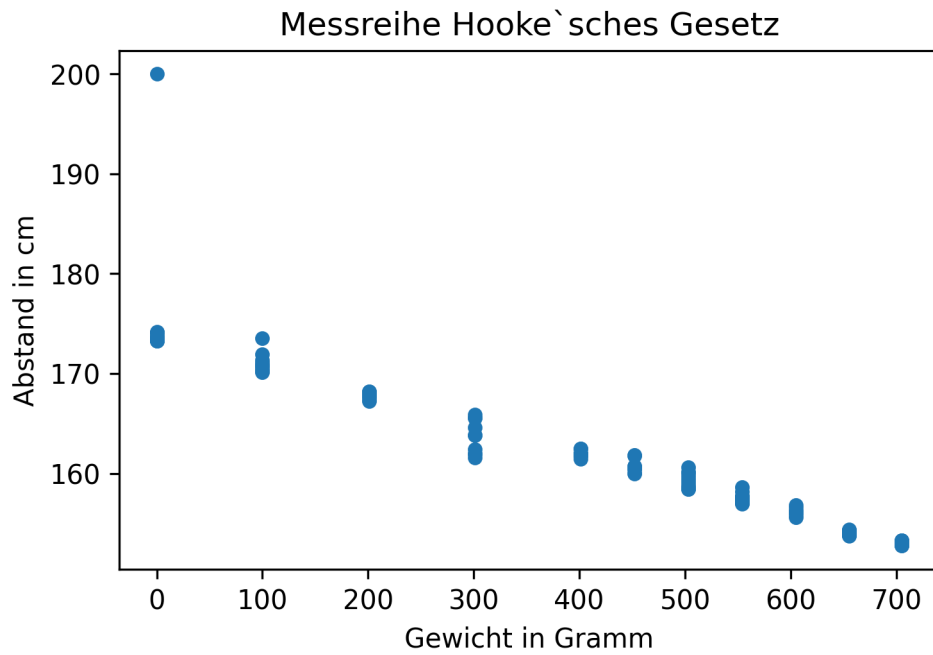
3.1.1 Deskriptive Statistik

```
print(hooke.head(), "\n")
print(hooke.tail())
```

```
hooke.describe()
```

```
hooke.groupby(by = 'mass')['distance'].describe()
```

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = "Messreihe Hooke`sches Geset
```



standardisiert in Einheiten der Standardabweichung (z-Werten)


```

gewicht = 0

# z-Transformation manuell berechnen
mittelwert_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].mean()
standardabweichung_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].std(ddof=1)

z_values = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].apply(lambda x: (x - mittelwert_ausdehnung) / standardabweichung_ausdehnung)
z_values.name = 'z-values'

# z-Transformation mit scipy
scipy_z_values = scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance'], ddof=1)
scipy_z_values.name = 'scipy z-values'

# gemeinsame Ausgabe der Daten
print(pd.concat([hooke[hooke['mass'] == gewicht], z_values, scipy_z_values], axis = 1))

```

Ausreißer

! Wichtig 4: Ausreißer

In der Statistik wird ein Messwert als Ausreißer bezeichnet, wenn dieser stark von der übrigen Messreihe abweicht. In einer Messreihe können auch mehrere Ausreißer auftreten. Diese Werte können zur Verbesserung der Schätzung aus der Messreihe entfernt werden, wenn anzunehmen ist, dass diese durch Messfehler und andere Störgrößen verursacht sind.

Eine Möglichkeit, Ausreißer zu identifizieren, ist die z-Transformation. Dabei muss ein Schwellenwert gewählt werden, ab dem ein Messwert als Ausreißer klassifiziert werden soll, bspw. 2,5 oder 3 Einheiten der Standardabweichung. In der Statistik wurde eine ganze Reihe von Ausreißertests entwickelt (siehe [Ausreißertests](#))

Die Einstufung eines Messwerts als Ausreißer kann aber nicht allein auf der Grundlage statistischer Verfahren erfolgen, sondern ist immer eine Ermessensentscheidung auf der Grundlage Ihres Fachwissens. Denn nicht alle abweichenden Werte sind automatisch ungültig, sondern treten mit einer gewissen statistischen Wahrscheinlichkeit auf (siehe Kapitel Normalverteilung). Man spricht dann von gültigen Extremwerten.

Ausreißer von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) und ist abrufbar auf [Wikipedia](#)

```
hooke.drop(index = 113, inplace = True)

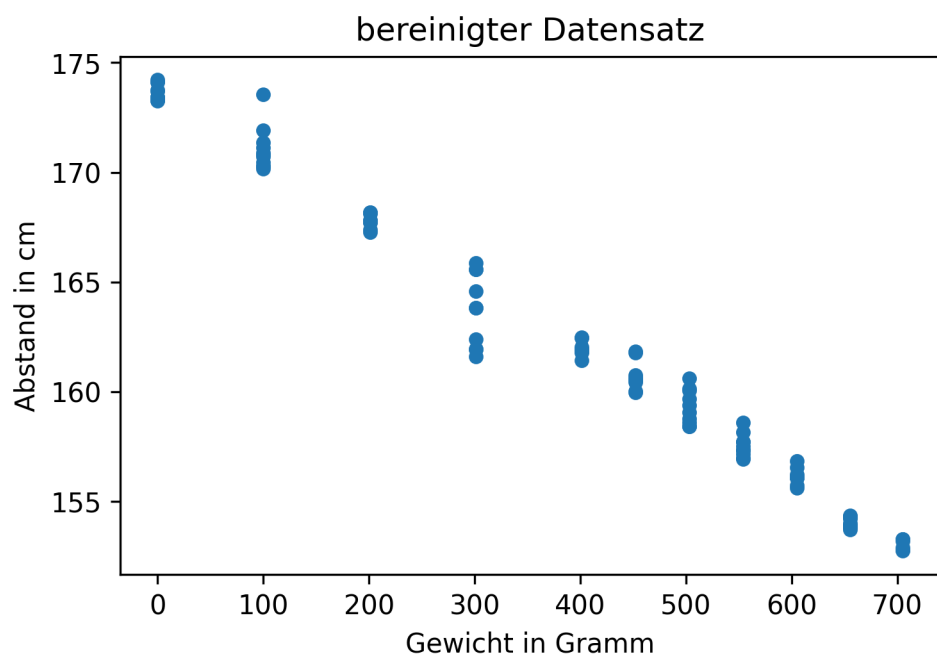
hooke.groupby(by = 'mass')['distance'].describe()
```

```
gewicht = 301

z_values = scipy_z_values = scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance'])
z_values.name = 'z-values'

print(pd.concat([hooke[hooke['mass'] == gewicht], z_values], axis = 1))
```

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter Datensatz', y1
```



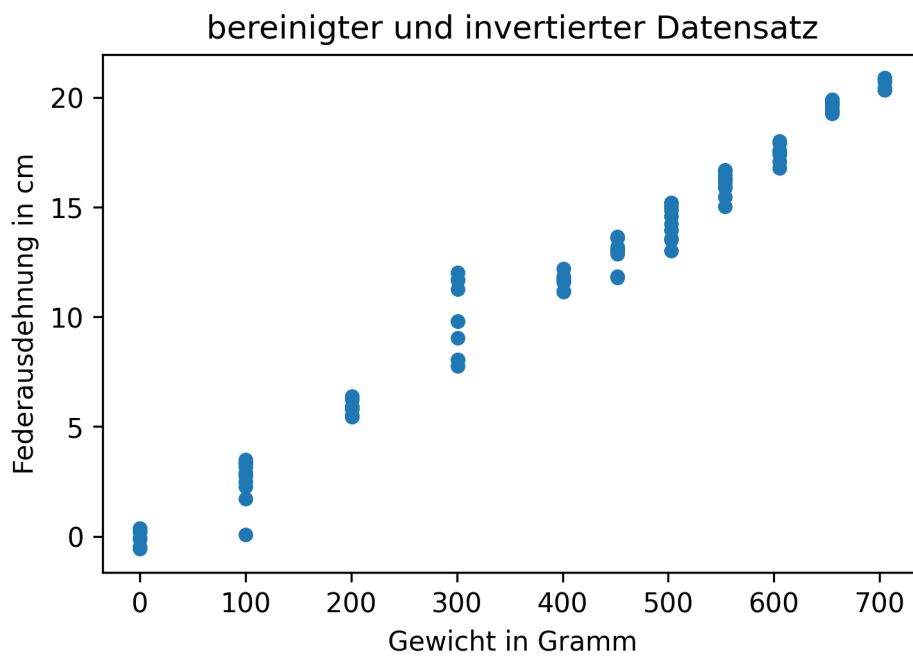
```

nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
print(f"Nullpunkt: {nullpunkt:.2f} cm")

hooke['distance'] = hooke['distance'].sub(nullpunkt).mul(-1)

hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter und invertierter Datensatz')

```



```

# Mittelwerte nach Gewicht
distance_means_by_weight = hooke['distance'].groupby(by = hooke['mass']).mean()
distance_means_by_weight.name = 'Federausdehnung'

```

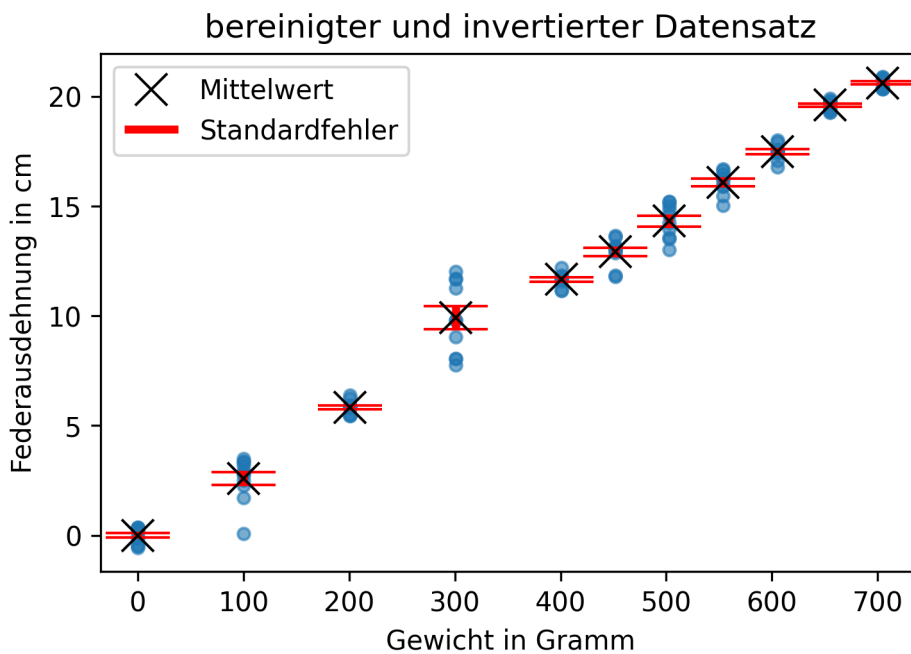
```
# Standardfehler nach Gewicht
distance_stderrors_by_weight = hooke['distance'].groupby(by = hooke['mass']).std(ddof = 1).d
distance_stderrors_by_weight.name = 'Standardfehler'

hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter und invertierter

errorbar_container = plt.errorbar(
    x = distance_means_by_weight.index, y = distance_means_by_weight, yerr = distance_stderrors
    linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor

# siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarC
plt.legend([errorbar_container.lines[0], errorbar_container.lines[2][0]],
           ['Mittelwert', 'Standardfehler'],
           loc = 'upper left')
plt.show()

print(pd.concat([distance_means_by_weight, distance_stderrors_by_weight], axis = 1))
```



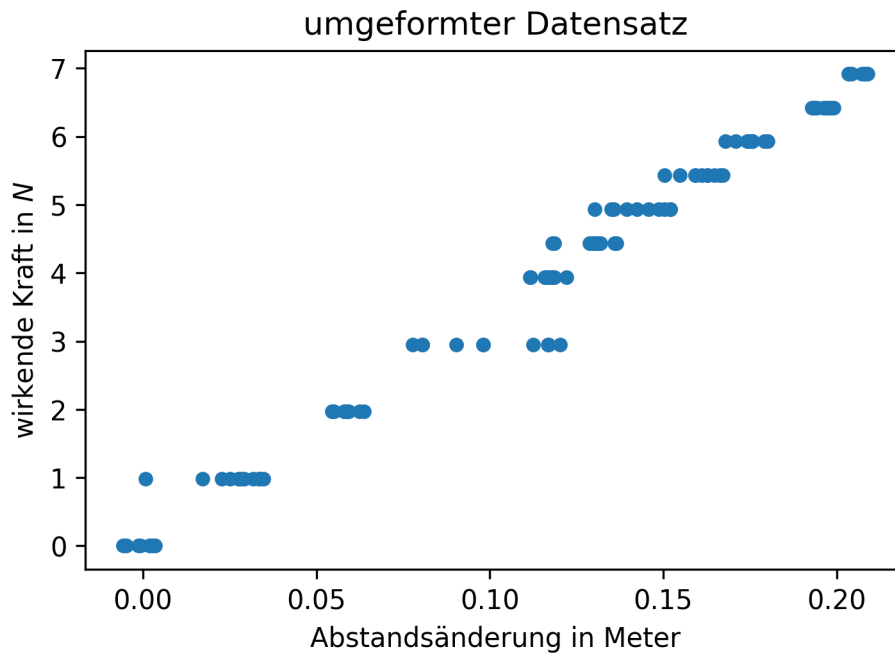
3.2 Federkonstante bestimmen

```
hooke['mass'] = hooke['mass'].div(1000).mul(9.81)
hooke.rename(columns = {'mass': 'force'}, inplace = True)

hooke['distance'] = hooke['distance'].div(100)

print(hooke.head())
```

```
hooke.plot(x = 'distance', y = 'force', kind = 'scatter', title = 'umgeformter Datensatz', y
```



3.2.1 Lineare Ausgleichsrechnung

[Wikipedia](#)

[drate](#)

[kleinsten Qua-](#)

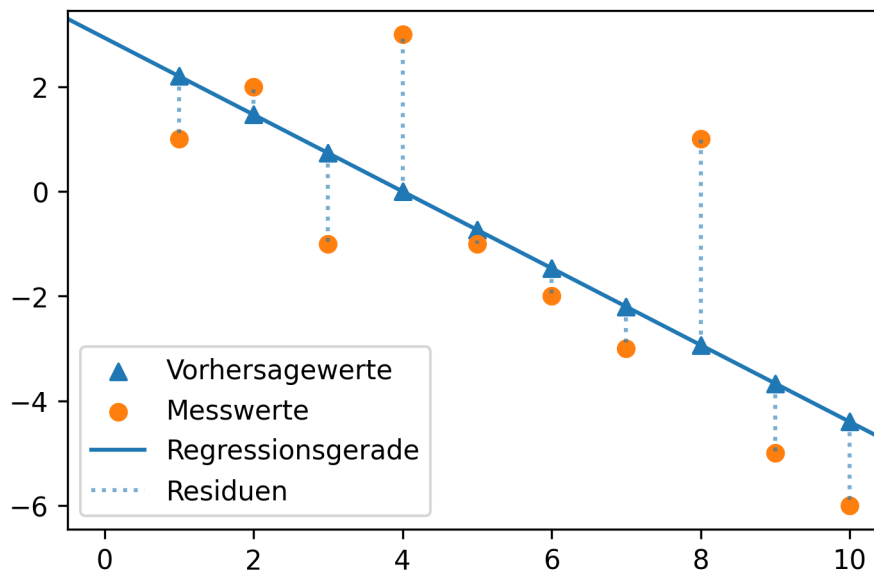
i Hinweis 5: Methode der kleinsten Quadrate

Mit der Methode der kleinsten Quadrate soll diejenige Gerade $\hat{y} = \beta_0 + \beta_1 \cdot x$ gefunden werden, die die quadrierten Abstände der Vorhersagewerte $\hat{y}$ von den tatsächlich gemessenen Werten y minimiert. Die Werte $y_i - \hat{y}_i$ sind die Residuen e_i . Es gilt also:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N e_i^2 = \min$$

Grafisch kann man sich die Minimierung der quadrierten Abstände so vorstellen.

3.3 Grafik



Regressionskoeffizienten: [2.93333333 -0.73333333]

3.4 Code


```

x = np.arange(1, 11)
y = - x.copy() + 4
y[0] -= 2
y[2] -= 2
y[3] += 3
y[-3] += 5

lm = poly.polyfit(x, y, 1)
vorhersagewerte = poly.polyval(x, lm)

plt.scatter(x, vorhersagewerte, label = 'Vorhersagewerte', marker = "^", color = "tab:blue")
plt.scatter(x, y, label = 'Messwerte', marker = 'o', color = "tab:orange")
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = "Regressionsgerade", color = "tab:blue")
dotted = plt.vlines(x, ymin = vorhersagewerte, ymax = y, alpha = 0.6, ls = 'dotted', label

plt.legend()
plt.show()

print("Regressionskoeffizienten:", lm)

```

Die eingezeichnete Gerade entspricht der linearen Funktion $\hat{y} = \beta_0 + \beta_1 \cdot x + e_i$. Die Dreiecksmarker sind die Vorhersagewerte $\hat{y}_i$ des linearen Modells für die Werte $x_i = np.arange(1, 11)$. Die tatsächlichen Messwerte y sind mit Kreismarkern markiert. Die Länge der gestrichelten Linien entspricht der Größe der Abweichung zwischen den Mess- und Vorhersagewerten $y_i - \hat{y}_i$, also den Residuen e_i .

Gesucht wird diejenige Gerade, die die Summe der quadrierten Residuen minimiert. Die gesuchten Werte β_0 und β_1 sind die Kleinst-Quadrat-Schätzer.

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

NumPy polyfit und polyeval

```
import numpy.polynomial.polynomial as poly
```

i Hinweis 6: polyfit und polyeval erklärt

```
# Beispieldaten erzeugen
x = np.array(list(range(0, 100)))
y = x ** 2

print(poly.polyfit(x, y, 1))
```

```
[-1617.    99.]
```

Die Funktion gibt die geschätzten Regressionsparameter als NumPy-Array zurück. Die Terme sind aufsteigend angeordnet, d. h. der Achsabschnitt steht an Indexposition 0, der Steigungskoeffizient an Indexposition 1. Die Ausgabe für ein Polynom zweiten Grades würde beispielsweise so aussehen:

```
print(poly.polyfit(x, y, 2))
```

```
[ 1.62413205e-12 -5.07904010e-14  1.00000000e+00]
```

Mit den Regressionskoeffizienten können die Vorhersagewerte der linearen Funktion berechnet werden. Dafür wird die Funktion `poly.polyeval(x, c)` verwendet. Diese berechnet die Funktionswerte für in `x` übergebene Wert(e) mit den Funktionsparametern `c`. Aus der Differenz der gemessenen Werte und der Vorhersagewerte können die Residuen bestimmt werden.

```
# 'manuelle' Berechnung
regressions_koeffizienten = poly.polyfit(x, y, 1)
vorhersagewerte = regressions_koeffizienten[0] + x * regressions_koeffizienten[1]
residuen = y - vorhersagewerte

# Berechnung mit polyeval
lm = poly.polyfit(x, y, 1)
vorhersagewerte_polyval = poly.polyval(x, lm)

print("Die Ergebnisse stimmen überein:", np.equal(vorhersagewerte, vorhersagewerte_polyval))
print("\nAusschnitt der Vorhersagewerte:", vorhersagewerte[:10])
```

Die Ergebnisse stimmen überein: True

Ausschnitt der Vorhersagewerte: [-1617. -1518. -1419. -1320. -1221. -1122. -1023. -924. -

Das [Bestimmtheitsmaß](#) R^2 gibt an, wie gut die Schätzfunktion an die Daten angepasst ist. Der Wertebereich reicht von 0 bis 1. Ein Wert von 1 bedeutet eine vollständige Anpassung. Für eine einfache lineare Regression mit nur einer erklärenden Variable kann das Bestimmtheitsmaß als Quadrat des [Bravais-Pearson-Korrelationskoeffizienten](#) r berechnet werden. Dieser wird mit der Funktion `np.corrcoef(x, y)` ermittelt (die eine Matrix der Korrelationskoeffizienten ausgibt).

```
print(f"r = {np.corrcoef(x, y)[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(x, y)[0, 1] ** 2:.2f}")
```

r = 0.97
 R^2 = 0.94

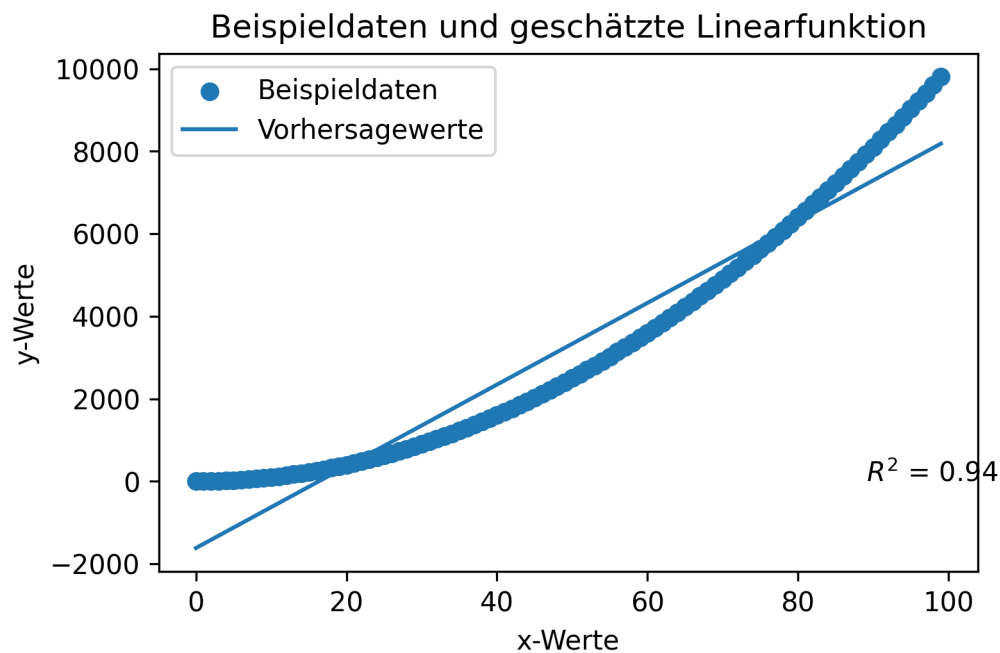
Die Daten und die geschätzte Gerade können grafisch dargestellt werden.

```
import matplotlib.pyplot as plt

plt.scatter(x, y, label = 'Beispieldaten')
plt.plot(x, vorhersagewerte, label = 'Vorhersagewerte')
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Beispieldaten und geschätzte Linearfunktion')
plt.xlabel('x-Werte')
plt.ylabel('y-Werte')
plt.legend()

plt.show()
```



i Hinweis 7: np.polyfit & np.polyval

Die Funktionen `np.polyfit(x, y, deg)` und `np.polyval(p, x)` funktionieren wie die vorgestellten Funktionen aus dem Modul `numpy.polynomial.polynomial`. Ein wichtiger Unterschied besteht jedoch darin, dass **die Parameter der Funktion `polyfit` in umgekehrter Reihenfolge** ausgegeben werden.

```
print(poly.polyfit(x, y, deg = 1))
print(np.polyfit(x, y, deg = 1))
```

```
[-1617.    99.]
[   99. -1617.]
```

i Hinweis

This forms part of the old polynomial API. Since version 1.4, the new polynomial API defined in `numpy.polynomial` is preferred. A summary of the differences can be found in the [transition guide](#).

[NumPy-Dokumentation](#)

```

print(poly.polyfit(hooke['distance'], hooke['force'], 1))

print(f"r = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1] ** 2:.2f}")

```

```

# Berechnung mit polyeval
lm = poly.polyfit(hooke['distance'], hooke['force'], 1)
vorhersagewerte_hooke = poly.polyval(hooke['distance'], lm)

```

```

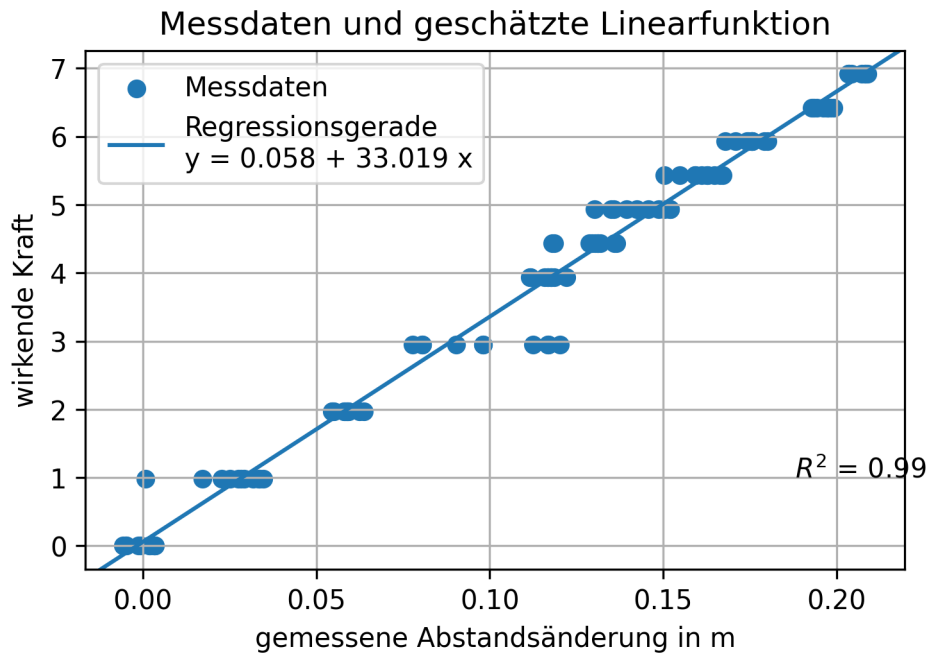
# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\
ny = ' + "{beta_0:.3f} + {beta_1:.3f}x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Messdaten und geschätzte Linearfunktion')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()

```



3.4.1 Messabweichung quantifizieren

[Wikipedia](#)

- [Residuenquadratsumme](#)
- [Summe der Abweichungsquadrate von \$x\$](#)
- [Summe der Abweichungsquadrate von \$y\$](#)

—

```

print(f"Regressionskoeffizient: {lm[1]:.4f}")

# 'manuell' Standardfehler des Regressionskoeffizienten berechnen
standardfehler_beta_1 = np.sqrt( (1 / (len(x) - 2) * sum((y - vorhersagewerte_hooke) ** 2)) )

print(f"Standardfehler des Regressionskoeffizienten: {standardfehler_beta_1:.4f}")

# Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n = len(x)

t_wert = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)
print(f"t-Wert 95%-Intervall (zweiseitig): {t_wert:.4f}")
print(f"Konfidenzintervall 95%: {lm[1]:.4f} ± {t_wert:.4f} * {standardfehler_beta_1:.4f}")
print(f"untere 95%-Intervallgrenze: {lm[1] - t_wert * standardfehler_beta_1:.4f}")
print(f"obere 95%-Intervallgrenze: {lm[1] + t_wert * standardfehler_beta_1:.4f}")

```

```

# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\
ny = ' + "{beta_0:.3f} + {beta_1:.3f}x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

# 95%-Konfidenzintervall einzeichnen

```

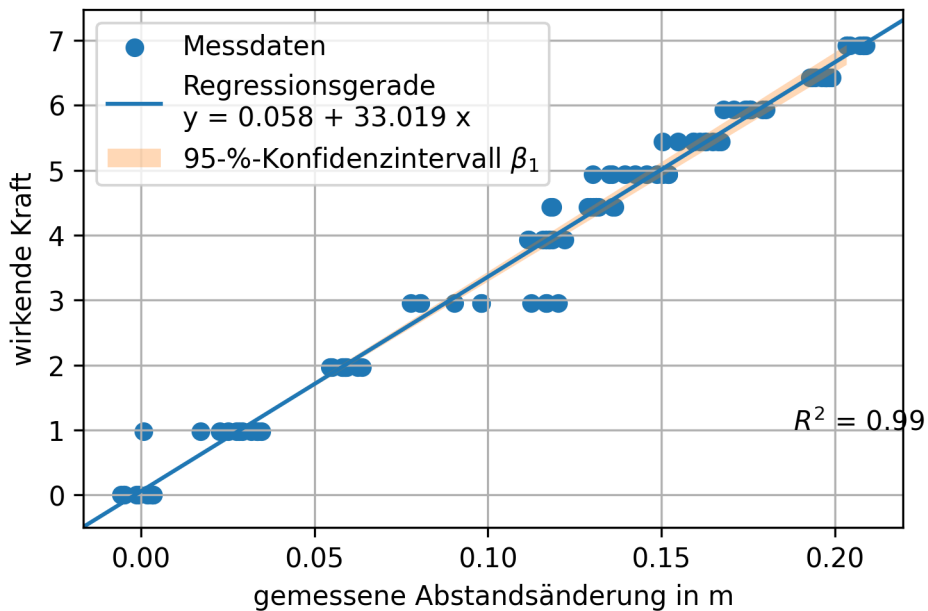
```
## poly.polyval(hooke['distance'], [lm[0]])
beta1_lower_boundary = lm[1] - (t_wert * standardfehler_beta_1)
beta1_upper_boundary = lm[1] + (t_wert * standardfehler_beta_1)

y_lower_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_lower_boundary])
y_upper_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_upper_boundary])

plt.fill_between(x = x, y1 = y_lower_boundary , y2 = y_upper_boundary, alpha = 0.3, label =

plt.title(label = 'Messdaten und geschätzte Linearfunktion im 95%-Intervall')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()
```



3.4.2 Das Modul SciPy

-
-
-
-
-
-

```
# Zuweisung mehrerer Objekte
slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

# Zuweisung eines Objekts
lm = scipy.stats.linregress(x, y)

print("\n", lm, sep = '')
print(f"y-Achsenschnittpunkt: {lm.intercept:.4f}\nStandardfehler des y-Achsenschnittpunkts:{"
```

```
alpha = 0.05
n = len(x)

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {slope +
```

3.4.3 Ergebnis Federkonstante

3.5 Resterampe

4 Fehlerrechnung

4.1 Fehlerarten und Umgang mit Messfehlern

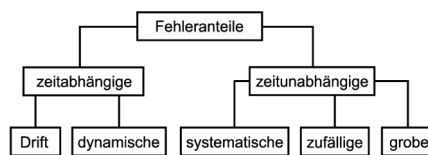


Abbildung 4.1: Fehlerquellen

-

-

Einschwingzeit

Quantifizierungsfehler

-

-

-

2016

2014

! Umgang mit verschiedenen Fehlerarten

1. Grobe Fehler: betroffene Werte streichen und Messung wiederholen.
2. Systematische Fehler: Systematische Fehler sollten, wenn sie bekannt sind, korrigiert (bspw. temperaturabhängige Genauigkeit des Messgeräts) oder quantifiziert (bspw. Genauigkeit des Ablesens) werden.
3. Zufällige Fehler: Die statistische Streuung von Messwerten um den Erwartungswert sollte quantifiziert werden.

4.2 Methoden zur Fehlerabschätzung

- 1.
- 2.
- 3.
- 4.

4.2.1 Fehlerabschätzung

-
-

Beispiel Messbedingungen

-
-

nur 2 Fingergliedknochen

Der Daumen besteht anatomisch aus

Messschieber mit Nonius

-
-

•

—

*

•

•

•

5 Übung Hooke'sches Gesetz

-
-
-
-
-

- 1.
- 2.
- 3.

5.1

5.2

-

—

-

-

5.1 Dateien einlesen

```
ordnerpfad = '01-daten/hooke'

pfadliste = glob.glob(pathname = '*', root_dir = ordnerpfad, recursive = False)
print(pfadliste)
print(f"Anzahl Dateien: {len(pfadliste)}")
```

```
list_of_files = []
for pfad in pfadliste:
    zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, nrows = 3)
    list_of_files.append(zwischenspeicher)
    print(pfad, "\n", zwischenspeicher, "\n", sep = '')
```


5.2 Aufgabe Dateien einlesen

-
-

a)
b)

c)

💡 Schrittweises Vorgehen

Das Einlesen der Dateien wird voraussichtlich der aufwändigste und fehleranfälligste Arbeitsschritt sein. Entwickeln Sie Ihre Lösung Schritt für Schritt. Beginnen Sie mit der Variante a). Wenn Sie die Dateien eingelesen haben, können Sie sich durch die Weiterentwicklung zur Variante b) mit dem Modul glob vertraut machen. Darauf aufbauend können Sie mit der Variante c) die Automatisierung für beliebig viele Dateien umsetzen.

💡 Musterlösung Dateien einlesen

Der erste Versuch, die Dateien einzulesen, scheitert mit einer Fehlermeldung. Die Anweisungen werden deshalb in die Struktur zur Ausnahmebehandlung eingebettet und die verursachende Datei abgefangen. (Sollten mehrere Dateien Fehler aufwerfen, müssten die Dateien in einer Liste gespeichert und später - falls möglich - mit einer Schleife weiter behandelt werden.)

```
hooke = pd.DataFrame(columns = ['Zeit', 'Abstand', 'Gewicht', 'Team']) # ein leerer DataFra

for pfad in pfadliste:

    try:
        zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, sep = '\t')

        # Dateiname als Spalte einfügen
        zwischenspeicher['Team'] = pfad[5:-4]

        hooke = pd.concat([hooke, zwischenspeicher], ignore_index = True)

    except Exception as error:
        print(pfad, error, sep = "\n")
        pfad_problem_datei = pfad

print(hooke.info(), "\n")
print("Erfolgreich einglesen:\n", hooke['Team'].unique(), sep = '')

team_ma.txt
Error tokenizing data. C error: Expected 3 fields in line 135, saw 4

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 359 entries, 0 to 358
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype

```

```

---  -----  -----  ----
0   Zeit      355 non-null  object
1   Abstand   355 non-null  object
2   Gewicht   355 non-null  float64
3   Team      359 non-null  object
dtypes: float64(1), object(3)
memory usage: 11.3+ KB
None

```

Erfolgreich einglesen:
['kreativkoepfe' 'die_ahnungslosen' 'fabi']

Der Fehlermeldung zufolge besteht Zeile 135 aus 4 statt aus 3 Spalten. Die fehlerverursachende Datei wird deshalb zeilenweise durchlaufen und jede Zeile ausgegeben, die mehr als 3 Einträge hat. Zur Kontrolle werden auch die ersten 5 Zeilen ausgegeben.

```

# einen leeren DataFrame mit 3 Spalten erstellen
df = pd.DataFrame(data = [], columns = ['Zeit', 'Abstand', 'Gewicht'])

dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

index = 0
for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            print("Index =", index, ":", zwischenspeicher)
        elif index <= 5:
            print(zeile)
        index += 1
    except Exception as error:
        print(error)

dateiobjekt_problem_datei.close()

```

```
10:09:38      109.26 cm    0
```

```
10:09:41      109.26 cm    0
```

```
10:09:44    109.28 cm    0
```

```
Index = 134 : ['10:29:27', '105.49 cm', '300', '\n']
```

Jede zweite Zeile ist leer. In Zeile 134 wird ein Zeilenumbruch ‘\n’ eingelesen. Die Datei wird deshalb mit einer angepassten Schleife erneut durchlaufen. Aus der betreffenden Zeile wird der zusätzliche Zeilenumbruch ‘\n’ entfernt. Leere Zeilen werden übersprungen. Die korrekten Zeilen werden an den DataFrame hooke angefügt.

Der Code muss ggf. noch angepasst werden, weil vermutlich so leere zeilen angefügt werden

```
dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            zwischenspeicher = zwischenspeicher[:3]
        elif len(zwischenspeicher) < 3: # leere Zeilen überspringen
            continue
        # Dateinamen anfügen
        zwischenspeicher.append(pfad[5:-4])

        hooke.loc[len(hooke)] = pd.Series(zwischenspeicher).values

    except Exception as error:
        print(error)
        print(pd.Series(zwischenspeicher).values)

dateiobjekt_problem_datei.close()

print("Erfolgreich einglesen:\n", hooke['Team'].unique(), "\n", sep = '')
print(hooke.info())
```

```
Erfolgreich einglesen:
```

```
['kreativkoepfe' 'die_ahnungslosen' 'fab']
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 537 entries, 0 to 536
```

```
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Zeit      533 non-null      object
1   Abstand   533 non-null      object
2   Gewicht    533 non-null      object
3   Team       537 non-null      object
dtypes: object(4)
memory usage: 21.0+ KB
None
```

Anschließend werden zum einen die Zeilen mit Nullwerten betrachtet ...

```
print(hooke.loc[hooke.apply(pd.isna).any(axis = 1), :])
```

	Zeit	Abstand	Gewicht	Team
3	NaN	NaN	NaN	kreativkoepfe
23	NaN	NaN	NaN	kreativkoepfe
28	NaN	NaN	NaN	kreativkoepfe
212	NaN	NaN	NaN	die_ahnungslosen

... und entfernt.

```
hooke.drop(np.where(hooke.apply(pd.isna).any(axis = 1))[0], inplace = True)
```

Zum anderen werden die Datentypen kontrolliert.

- Die Zeit kann als string stehen bleiben, da sie für die Auswertung nicht benötigt wird.
- Der gemessene Abstand ist mit ' cm' notiert - diese Zeichenkette wird entfernt. Anschließend sollte die Spalte als numerisch erkannt werden.
- Das Gewicht sollte numerische Werte enthalten, wird aber als Datentyp object eingelesen und muss weiter untersucht werden.
- Der Spalte Team könnte der [Pandas Datentyp category](#) zugewiesen werden, notwendig ist es aber nicht.

String ' cm' entfernen.

```
hooke.replace(' cm', '', regex = True, inplace = True)
```

Ob alle Elemente einer Zelle numerisch sind, kann mit der Pandas-Methode `pd.Series.str.isnumeric()` überprüft werden. Ein Blick auf die Daten zeigt die Ursache.

```
print(hooke['Gewicht'].str.isnumeric().sum())
```

```
print(hooke['Gewicht'].head())
```

```
print(hooke['Gewicht'].tail())
```

```
1
0    0.0
1    0.0
2    0.0
4    0.0
5    0.0
Name: Gewicht, dtype: object
532    850\n
533    850\n
534    850\n
535    850\n
536    850\n
Name: Gewicht, dtype: object
```

Die Zeilenumbrüche werden ebenfalls entfernt.

```
hooke.replace('\n', '', regex = True, inplace = True)
```

```
print(hooke['Gewicht'].str.isnumeric().sum())
```

```
print(hooke['Gewicht'].tail())
```

```
178
532    850
533    850
534    850
535    850
536    850
Name: Gewicht, dtype: object
```

```
print(hooke.info(), "\n")
```

```
# explizite Zuweisung
```

```
hooke['Abstand'] = hooke['Abstand'].astype('float')
```

```
hooke['Gewicht'] = hooke['Gewicht'].astype('float')
```

```
print(hooke.info())
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Zeit       533 non-null    object
1   Abstand    533 non-null    object
2   Gewicht    533 non-null    object
3   Team       533 non-null    object
dtypes: object(4)
memory usage: 20.8+ KB
None

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Zeit       533 non-null    object
1   Abstand    533 non-null    float64
2   Gewicht    533 non-null    float64
3   Team       533 non-null    object
dtypes: float64(2), object(2)
memory usage: 20.8+ KB
None

```

```

hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].describe()

```


5.3 Daten aufbereiten

-
-
-

- 1.

5.3.1 Auf Ausreißer prüfen

5.4 Ausreißer bestimmen

5.5 Gemeinsame Ausgabe der Messreihen

5.6 Code

```
# z-Werte größer gleich abs(3) finden
z_values_ge3_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))
print("Anzahl der studentisierten z-Werte mit Betrag >= 3:", z_values_ge3_sum)

# z-Werte größer gleich abs(2.5) finden
z_values_ge25_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))
print("Anzahl der studentisierten z-Werte mit Betrag >= 2.5:", z_values_ge25_sum)

# Die Zeilen mit z-Werten größer abs(2.5) ausgeben
bool_index = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x) >= 2.5)
z_values_ge_25 = hooke.iloc[bool_index, :]
print(z_values_ge_25, "\n")

# Kombinationen aus Gewicht & Team bestimmen

teams = z_values_ge_25['Team'].unique()

## teams durchlaufen und jeweils die Gewichte speichern
team_gewichte = [] # leere liste
for i in range(len(teams)):
    print(teams[i])
    print(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'], "\n")
    team_gewichte.append(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'].values)

print(team_gewichte, "\n")

# Messreihen auswählen
messreihen = pd.DataFrame()
for i in range(len(teams)):
    for j in range(len(team_gewichte[i])):
        print(teams[i], team_gewichte[i][j])

    messreihen = pd.concat([messreihen, hooke.loc[ (hooke['Team'] == teams[i]) & (hooke['Gewicht'] == team_gewichte[i][j]) ]])
```

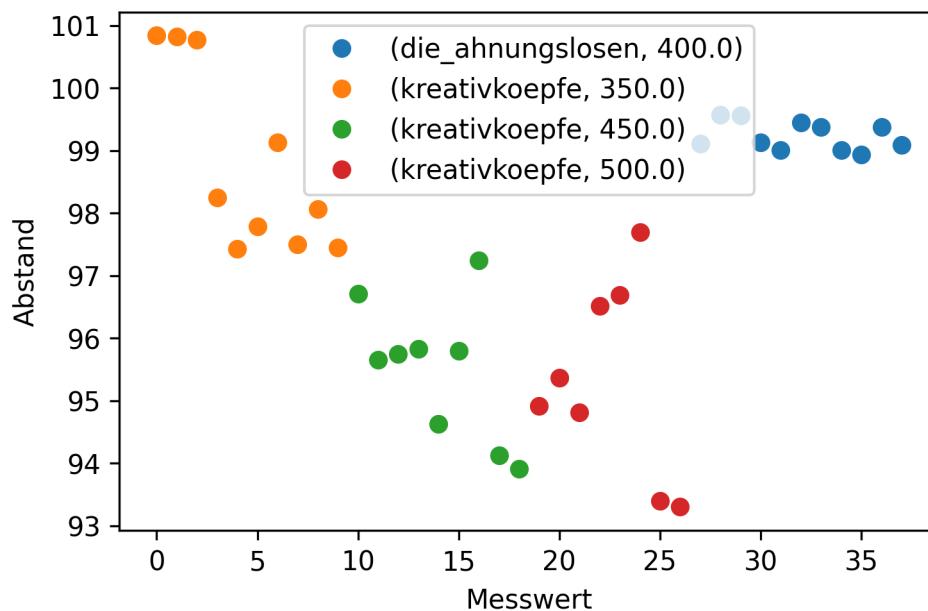
```
# studentisierte z-Werte der Messreihen bilden
messreihen_z_scores = messreihen.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x

# gemeinsame Ausgabe der Daten
messreihen.insert(loc = 2, column = 'z-Werte Abstand', value = messreihen_z_scores.values)
print(messreihen)
```

```
messreihen.reset_index(drop = True).groupby(by = ['Team', 'Gewicht'])['Abstand'].plot(marker

plt.xlabel('Messwert')
plt.ylabel('Abstand')
plt.legend()

plt.show()
```



5.6.1 Umwandlung der Rechengrößen

Tipp 3: Musterlösung

Abstandsmessung auf Meter und auf den Nullpunkt normieren

Abstandsmessung auf den Nullpunkt normieren. Die Spalte Abstand wird in Abstandsänderung umbenannt.

```
nullpunkte = hooke.loc[hooke['Gewicht'] == 0, :].groupby(by = 'Team')['Abstand'].mean()
print(nullpunkte)

teams = nullpunkte.index

for i in range(len(teams)):

    hooke.loc[hooke['Team'] == teams[i], 'Abstand'] = hooke.loc[hooke['Team'] == teams[i], :].mean()

hooke.rename(columns = {'Abstand': 'Abstandsänderung'}, inplace = True)
hooke['Abstandsänderung'] = hooke['Abstandsänderung'].div(100)
```

```
Team
die_ahnungslosen    109.759000
fabi                 109.812000
kreativkoepfe       109.676667
Name: Abstand, dtype: float64
```

Gewicht in wirkende Kraft umrechnen

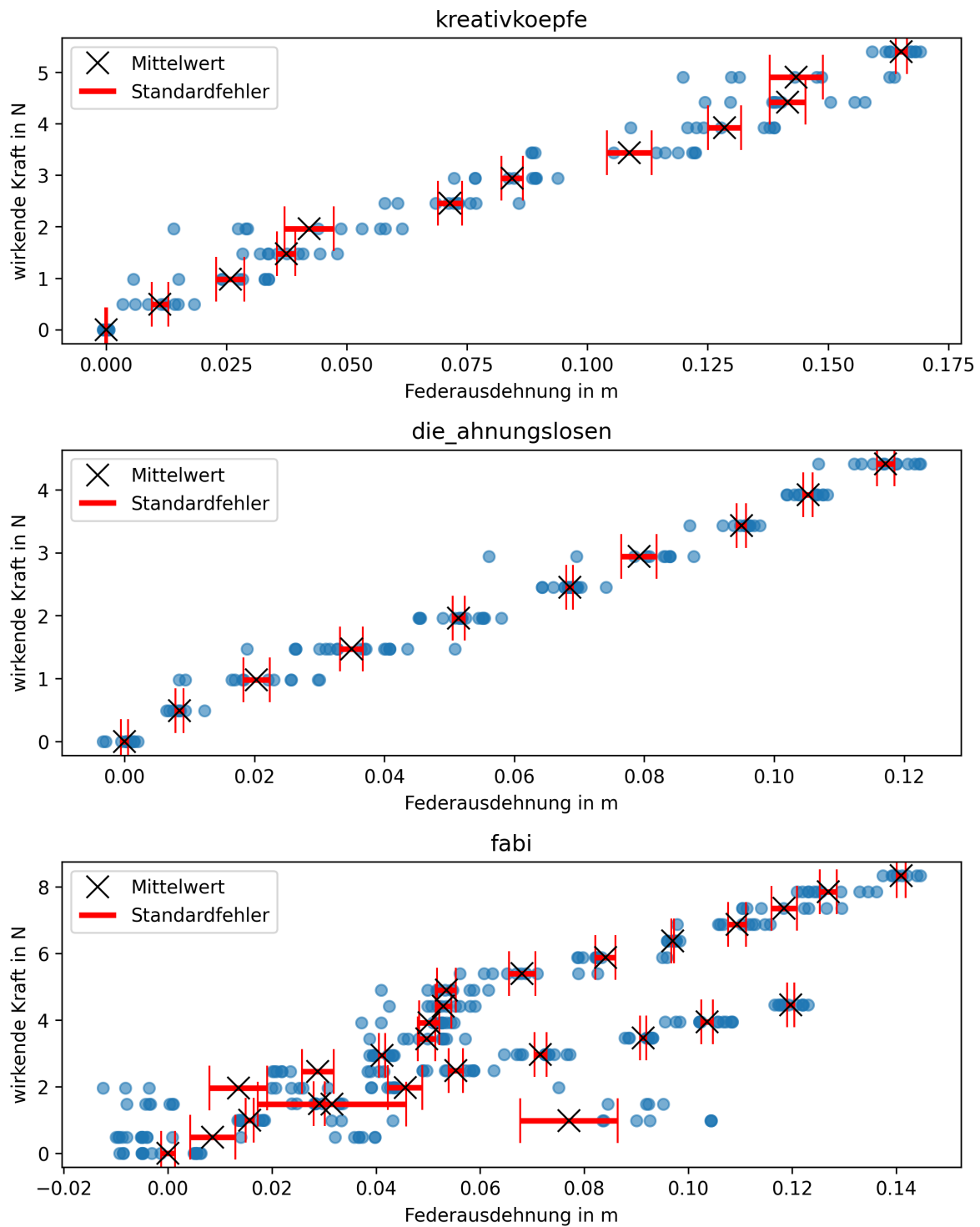
Gewicht in g in die wirkende Kraft in N umrechnen. Die Spalte wird in den Datensatz eingefügt. umbenannt.

```
hooke['Kraft'] = hooke['Gewicht'].div(1000).mul(9.81)
```

```
hooke.groupby(by = ['Team', 'Kraft'])['Abstandsänderung'].describe()
```


5.6.2 Grafische Darstellung

5.7 Grafische Darstellung



5.8 Mittelwerte und Standardfehler berechnen

5.9 Code

```
# Mittelwerte der Teams nach Kraft
distance_means_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']])['Abstand'].mean()
distance_means_by_team_and_force.name = 'Federausdehnung'

print(distance_means_by_team_and_force.head())

# Standardfehler der Teams nach Kraft
distance_stderrors_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']])['Abstand'].std()
distance_stderrors_by_team_and_force.name = 'Standardfehler'

print(distance_stderrors_by_team_and_force.head())

# grafische Darstellung
anzahl_teams = hooke['Team'].unique().size

plt.figure(figsize = (7.5, 12))
for i in range(anzahl_teams):

    plt.subplot(4, 1, i + 1) # plt.subplot zählt ab 1
```

```

# Punktdiagramm
plotting_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
plt.scatter(x = plotting_data['Abstandsänderung'], y = plotting_data['Kraft'], alpha = 0.6)

plt.title(label = hooke['Team'].unique()[i])
plt.xlabel("Federausdehnung in m")
plt.ylabel("wirkende Kraft in N")

# # Fehlerbalken
distance_means_by_force = plotting_data.groupby(by = plotting_data['Kraft'])['Abstandsänderung'].mean()
distance_stderrors_by_force = plotting_data.groupby(by = plotting_data['Kraft'])['Abstandsänderung'].std()

errorbar_container = plt.errorbar(x = distance_means_by_force, y = distance_means_by_force,
linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor = 'black')

# siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer
plt.legend([errorbar_container.lines[0], errorbar_container.lines[2][0]],
           ['Mittelwert', 'Standardfehler'],
           loc = 'upper left')

plt.tight_layout()
plt.show()

```

5.10 Auswertung

-
-
-
-

```

hooke.drop(index = hooke.loc[hooke['Team'] == 'ma', :].index, inplace = True)
hooke.drop(index = hooke.loc[(hooke['Team'] == 'fabi') & (hooke['Gewicht'] == 50), :].index,

```

 Tipp 4: Vorgehen bei vielen Datensätzen

Bei einer großen Anzahl an Datensätzen kann auch die grafische Kontrolle an Grenzen stoßen. In diesem Fall empfiehlt es sich, die visuelle und kennzahlenbasierende Methoden zusammen zu nutzen, um Muster zu identifizieren und für eine große Zahl von Messungen zu überprüfen. Beispielsweise könnten nach einer visuellen Inspektion von Messreihen mit Extremwerten bzw. Ausreißern alle Messreihen daraufhin überprüft werden, ob mit zunehmenden Gewicht stets auch die mittlere Federausdehnung größer als für leichtere Gewichte ist. Abweichende Messreihen könnten dann grafisch kontrolliert werden.

5.11 Federkonstanten bestimmen

2.

3.

5.12 $\alpha = 0.10$

5.13 $\alpha = 0.05$

5.14 Code

```
anzahl_teams = hooke['Team'].unique().size
alpha = 0.10

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)

    print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

    slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
    print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
          f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
          f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')
```

```

    print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

alpha = 0.05

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)

    print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

    slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
    print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
          f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
          f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

    print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

```

5.15 Auswertung

Warning 2: Ergebnisse

Abhängig vom gewählten Vorgehen sind andere Ergebnisse möglich, beispielsweise durch das Entfernen von als Ausreißern eingestuft Einzelwerten oder einer anderen Behandlung der Messreihe vom Team fabi für das angehängte Gewicht 50 Gramm.

Quellen

3-658-06114-2

10.1007/978-

<https://www.tu-braunschweig.de/index.php?eID=dumpFile&t=f&f=45992&token=b041478add5f7a3e136b69905b72bfa0e192d82b>